arXiv:2607.11388v1 [cs.AI] 13 Jul 2026

StructAgent: Harness Long-horizon Digital Agents with Unified Causal Structure

Wenyi Wu[1,2,§], **Sibo Zhu**[1,2,§], **Kun Zhou**[2,†], **Aayush Salvi**[1], **Zixuan Song**[1], **Biwei Huang**[1,2]
[1]University of California, San Diego [2]Aether AI Lab
[§]Work done during an internship at Aether AI.
[†]Project Lead & Corresponding Author: franciskunzhou@gmail.com.

Abstract

Recent advances in large language models (LLMs) and vision-language models (VLMs) have enabled increasingly capable digital agents for computer use. However, real-world tasks are often long-horizon and involve evolving contexts containing accumulated observations, intermediate edits, failed attempts, and partially completed executions. Existing agents typically operate over raw interaction history, making task progress difficult to interpret, verify, and recover, which ultimately limits reliable long-horizon execution. In this paper, we argue that addressing this challenge requires explicitly structuring both the agent's state and workflow around a unified causal representation of task progress. We present **StructAgent**, a state-centered framework that introduces a unified state for maintaining compact, verifiable task progress and a structured workflow that regulates progress through verifier-backed state transitions. Building on this design, StructAgent further enables explicit progress checkpointing, evidence-driven task completion, targeted failure recovery, and tool-supported execution, while ensuring that all progress updates remain grounded in verification. Extensive experiments demonstrate that StructAgent consistently improves a wide range of LLM and VLM backbones on long-horizon computer-use tasks. On OSWorld-Verified, it improves Qwen3.5-9B from 27.0% to 46.9% success rate and Qwen3.5-27B from 31.6% to 62.2%, while achieving a new open-source state of the art of 78.9% with MiniMax-M3. Moreover, the same framework generalizes beyond desktop environments to Minecraft, demonstrating the generality of our design.

Project Page **Code**

"The purpose of abstracting is not to be vague, but to create a new semantic level in which one can be absolutely precise." — Edsger W. Dijkstra

1 Introduction

Driven by scaling laws, large language models (LLMs) and vision-language models (VLMs) have become increasingly capable, demonstrating stronger abilities in perception, reasoning, and planning [1, 2]. Building on these advances, digital agents have emerged as a promising paradigm for automating user tasks in computer environments [3, 4]. It completes user requests by observing the digital environment through visual screenshots or accessibility information, taking actions such as clicking, typing, and invoking system tools when necessary [5–8].

Preprint.

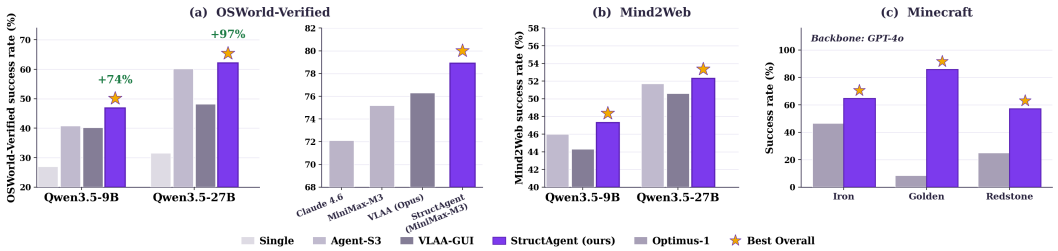


Figure 1: Results on OSWorld-Verified, Mind2Web, and Minecraft. StructAgent improves matched open backbones and reaches **78.9%** with MiniMax-M3, the state-of-the-art open-source model result.

In real-world applications, however, digital agents must execute long-horizon, complex workflows involving information retrieval, document editing, cross-application coordination, while preserving intermediate results across many steps. This setting is particularly challenging because the agent must continually manage an evolving context that accumulates large amounts of factual information, intermediate edits, failed attempts, and partially completed executions. As this context grows, critical information can become buried and the current task state increasingly ambiguous. Consequently, the agent struggles to maintain a clear causal understanding of task progress, making it difficult to promptly detect abnormal states, identify intermediate errors, and perform accurate planning and execution [8, 9].

To solve it, the agent's working process must be explicitly organized to support tracing the effects of individual decisions, identifying the key factors that drive task progress, and retaining the minimum sufficient context for future execution. Therefore, we focus on two fundamental elements of long-horizon agents: **State** and **Workflow**. Our goal is to endow both with a unified, minimum-sufficient causal structure that provides a common foundation for planning, acting, and verification. To this end, we introduce two complementary components: a unified state, which provides all modules with a shared interface for reading and writing, and a structured workflow, which governs how progress is propagated across components through transparent state transitions. Concretely, the unified state consists of current requirements, useful values, and verified evidences, enabling all modules to access a consistent and verifiable representation of task progress. The structured workflow establishes a fixed execution loop that iteratively invokes the planner, actor, and verifier to drive state updates, thereby preserving a transparent causal chain throughout long-horizon execution.

Building on this design, we introduce **StructAgent**, a state-centered framework for long-horizon computer use. Beyond organizing planning, acting, and verification into a unified control loop, owing to the unified state and structured workflow designs, StructAgent enables explicit progress checkpointing through verifier-backed state updates, evidence-driven task completion instead of self-reported DONE claims, targeted failure recovery based on verified evidences, and tool-supported execution that assists execution without bypassing state verification. Together, these mechanisms make long-horizon execution transparent, controllable, and robust.

Empirically, our agent framework is general to improve different LLM and VLM backbones on computer use tasks. On OSWorld-Verified, StructAgent improves Qwen3.5-9B from 27.0% to 46.9% success rate and Qwen3.5-27B from 31.6% to 62.2%. Using MiniMax-M3 backbone [10], StructAgent reaches 78.9%, achieving a new state-of-the-art performance among open-source methods on OSWorld-Verified [4], as summarized in Figure 1. A Minecraft instantiation shows that our agent framework can be applied beyond desktop GUI benchmarks and lead to significant improvement, by replacing desktop evidence sources with inventory-based verification.

We summarize our contributions as follows:

1. Unified state. We introduce a compact, typed, and auditable state that provides planning, acting, and verification with a shared representation of task progress.

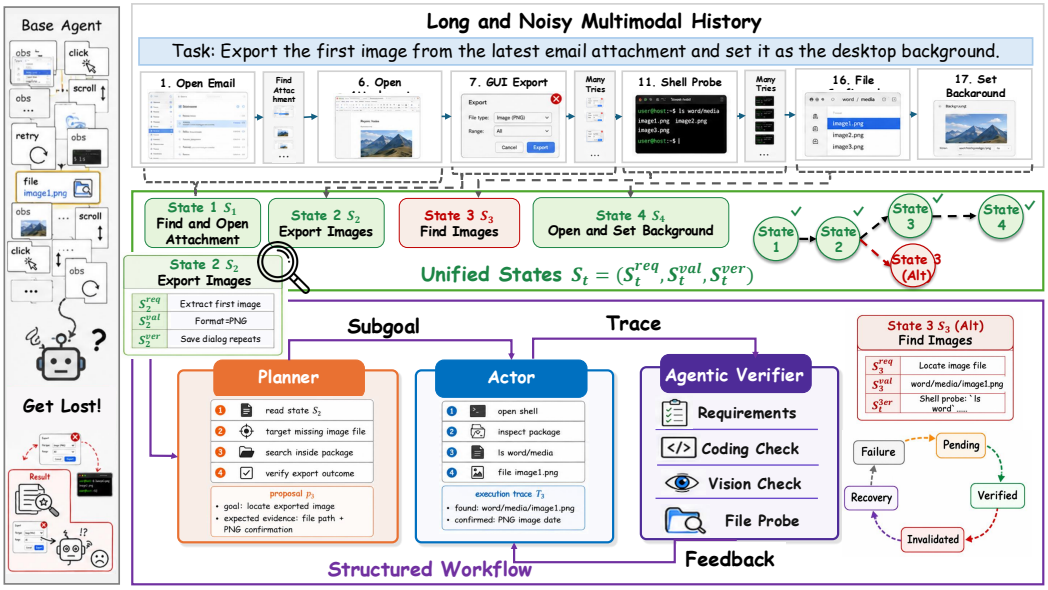


Figure 2: StructAgent overview. A unified state provides the shared progress interface, while a structured workflow controls how planning, acting, and verification interact with it. Planner and actor outputs can propose progress, but only verifier-backed decisions commit or invalidate state.

2. Structured workflow. We design a planning-acting-verification loop in which only verifier-backed decisions can commit, preserve, or invalidate progress, while memory, tools, and recovery provide state-conditioned support.

3. Broad empirical validation. StructAgent achieves a new open-model state-of-the-art on OSWorld with MiniMax-M3, and generalizes the same structured state to web and Minecraft environments.

2 Preliminaries

Digital Agent Task. A digital agent aims to automate computer-use tasks by interacting with a digital environment to fulfill a user instruction u. At each interaction step t, the agent receives an observation o_t, which may consist of a screenshot, accessibility information, or tool feedback, and selects the next action according to

$$a_t \sim \pi_\theta(\cdot \mid u, o_t), \qquad o_{t+1} = \mathcal{E}(o_t, a_t), \tag{1}$$

where $a_t \in \mathcal{A}$, and the action space $\mathcal{A}$ includes GUI operations (e.g., clicking, typing, scrolling, and waiting) as well as available system and application tools. Here, π_θ denotes the agent policy, and $\mathcal{E}$ is the environment transition induced by executing a_t. The interaction continues until the environment verifies that the execution satisfies the user instruction, at which point the task is considered successful.

Agent Context Management. Before t-th turn, the agent accumulates a raw interaction context $[o_1, a_1, \cdots, o_t, a_t]$, together with intermediate outputs, tool feedback, and environment responses. As the trajectory grows, the context increasingly mixes task-relevant facts with failed attempts, transient observations, intermediate edits, and unsupported completion claims. Effective long-horizon execution therefore requires the agent to distinguish past events from the information that remains valid and useful for future decisions. StructAgent addresses this by re-organizing the raw interaction history into an internal state s_t, which retains the compact, persistent, and verifiable information for planning, acting, and verification.

3 StructAgent

To better support digital agents in long-horizon tasks, we propose StructAgent, a general framework for structured agent context management. StructAgent introduces a unified causal structure composed of a unified state and a structured workflow. Together, these components enable more transparent and controllable planning and execution, while naturally supporting advanced capabilities such as progress checkpointing and targeted failure recovery.

3.1 Unified agent state

The unified agent state aims to turn a long and noisy interaction history into a compact state representation s_t that planning, acting, and verification can share. This makes s_t both the shared interface for reading task progress and the explicit causal surface through which progress can be updated.

State schema. Instead of asking each module to infer progress from raw trajectories, we maintain a state s_t that records what the current task still requires, what information should be carried forward, and what evidence supports the current progress:

$$s_t = \left(s_t^{\text{req}}, s_t^{\text{val}}, s_t^{\text{ver}}\right), \tag{2}$$

where s_t^{req} stores current requirements, s_t^{val} stores useful values, and s_t^{ver} stores verified evidences. In this way, s_t retains only the minimum sufficient information required to guide future planning and execution. Concretely, s_t^{req} stores the requirements of the current subgoal, specifying what must be achieved and what the verifier should check. s_t^{val} stores useful values discovered during execution, such as file paths, URLs, selected entities, and extracted fields. s_t^{ver} stores verified evidences collected by the verifier during the agentic probing process, so later turns can audit and reuse this information.

Causality-oriented state transition. To ensure that the state s_t faithfully reflects the true task progress, we design a causality-oriented state transition mechanism. In StructAgent, every state transition must be triggered by an explicit verifier-backed decision. Consequently, each progress update has a clear causal dependency on verifiable evidence, making the evolution of the state transparent and auditable. Concretely, each requirement $r_i \in s_t^{\text{req}}$ is associated with a status $\sigma_{i,t} \in \{\textsc{Pending}, \textsc{Verified}, \textsc{Invalidated}\}$, which respectively indicates that the requirement is not confirmed, verified with sufficient verified evidences, or invalidated by conflicting verified evidences. After the actor completes a subgoal, the verifier examines the task-relevant evidence and produces a verification decision d_t, determining whether each affected requirement should be verified, invalidated, or left unchanged. The verification decision then drives the state transition according to

$$\sigma_{i,t+1} = \begin{cases} \textsc{Verified}, & d_t \text{ verifies } r_i, \\ \textsc{Invalidated}, & d_t \text{ invalidates } r_i, \\ \sigma_{i,t}, & \text{otherwise.} \end{cases} \tag{3}$$

It ensures that only verified evidences can advance or revoke task progress. If the verifier additionally identifies useful values required by subsequent execution, such as file paths, URLs, and selected entities, they are written into s_{t+1}^{val} together with their supporting verified evidences in s_{t+1}^{ver}.

3.2 Structured agent workflow

Using the current state s_t, StructAgent runs a planning-acting-verification workflow:

$$s_t \xrightarrow{\text{Planner}} g_t \xrightarrow{\text{Actor}} \tau_t \xrightarrow{\text{Verifier}} d_t \xrightarrow{\text{Update}} s_{t+1}. \tag{4}$$

Here g_t is the proposed subgoal, τ_t is the execution trace, and d_t is the state update decision. The workflow is state-centric: planning reads state to decide what to try, acting attempts the subgoal in the environment, and verification decides what can be written back into state.

Planner. The planner reads s_t and selects the next subgoal g_t. Because the state already exposes current requirements, useful values, and verified evidences, the planner does not need to reconstruct progress from raw history.

Actor. The actor receives g_t together with the current state and attempts the subgoal in the environment. It predicts the action trace τ_t, which contains the next actions taken. Then, the action trace is executed and returns the resulting environment feedback.

Verifier. The verifier checks whether the attempted subgoal produced valid progress. It uses structured and agentic probing to examine task-relevant evidence, then outputs the state-update decision d_t. This decision may verify progress, invalidate earlier progress, or keep the relevant state unchanged.

Memory. We add the memory module as the state-conditioned support. It may suggest useful subgoals, action patterns, or verification checks. Reusable experience helps decide in the above workflow, including the next subgoal and action prediction, and the evidence verification.

3.3 Controllable Long-horizon Agentic Planning

The above design naturally makes StructAgent more transparent and controllable, enabling advanced capabilities such as progress checkpointing, failure recovery, and tool-supported execution.

Progress Checkpointing and Resuming. The state history naturally serves as a reusable checkpoint for long-horizon execution. Specifically, s_t^{req} records the minimum sufficient set of current requirements needed to characterize the current task progress, allowing the agent to resume execution without replaying the full interaction history. Meanwhile, s_t^{val} preserves useful values required by downstream steps, and s_t^{ver} stores verified evidences supporting verified progress. Together, these components provide a self-contained, human-readable snapshot of the agent's execution state, enabling efficient and reliable workflow resumption.

Failure Recovery and Human Takeover. When the state fails to advance, StructAgent recovers from s_t^{ver} rather than relying on a generic retry loop. The verified evidences identify whether the previous attempt lacked sufficient evidence, conflicted with a verified requirement, repeatedly failed under the current strategy, or was blocked by the environment. Based on this diagnosis, the system selects a targeted intervention, such as replanning, retrying execution, requesting an additional check, repairing the environment, or escalating the task for human takeover when necessary. The selected intervention is inserted as a new subgoal within the same workflow, and any subsequent progress is still committed only through a verifier-backed decision.

Tool-supported execution. StructAgent can identify recurring execution patterns from memory and interaction history and summarize them into reusable procedures. In the current runtime, these procedures are exposed through structured memory and typed execution surfaces, each associated with the subgoals it can support, its execution constraints, and the evidence required to verify its outcome. At runtime, the actor can retrieve and invoke these state-conditioned supports to execute g_t more efficiently and reliably, while only verifier-backed decisions can commit their outcomes as valid progress.

4 Experiments

4.1 Benchmarks and Setup

Benchmarks. Our main evaluation uses OSWorld-Verified [4], the verified split of OSWorld [3] for long-horizon computer use in real desktop applications. We report the aggregated results in the main table and provide the full per-domain breakdown in Appendix A. We also evaluate Mind2Web [11] as a web-generalization protocol, grouped by website category. Mind2Web is reported separately because it uses judge-based trajectory grading rather than executable desktop task scorers. Finally, we

Table 1: OSWorld-Verified main results. Success rate (%) is computed as mean Success/Total. The first three rows are reported leaderboard single-model results, the next three are reported framework results, and the remaining rows are self-run comparisons grouped by open-source backbone. Best values within each reported block or self-run backbone group are in **bold**. Office = Calc/Impress/Writer; Daily = Chrome/Thunderbird/VLC; Prof. = GIMP/VS Code.

System	OS	Office	Daily	Prof.	Multi-Apps	Overall
Claude Sonnet 4.6	**91.7**	75.2	76.8	70.8	**60.2**	72.1
Kimi K2.6	79.2	80.0	**77.1**	**81.6**	55.0	73.1
Qwen3.7-Plus	79.2	**82.9**	74.0	73.5	59.2	**73.3**
Agent S3 (Opus/GPT, bBoN)	79.2	81.1	64.9	77.6	**63.9**	72.6
OS-Symphony (GPT-5)	78.3	65.7	65.9	79.6	55.3	65.8
VLAA-GUI (Opus 4.5)	**91.7**	**84.3**	**72.8**	**83.7**	61.1	**76.3**
Qwen3.5-9B						
Single model	41.7	20.5	41.0	31.2	17.7	27.0
Agent S3	54.2	29.9	**58.6**	60.4	26.6	40.8
OS-Symphony	56.5	33.0	55.8	58.3	**32.6**	43.3
VLAA-GUI	**69.6**	27.8	47.9	**66.7**	28.5	40.2
StructAgent (ours)	62.5	**51.2**	51.8	58.3	27.5	**46.9**
Qwen3.5-27B						
Single model	50.0	26.1	46.0	37.5	18.3	31.6
Agent S3	70.8	60.6	59.5	66.7	**54.0**	60.2
OS-Symphony	70.8	47.9	58.4	64.6	43.5	52.8
VLAA-GUI	**83.3**	30.7	56.1	62.5	47.3	48.2
StructAgent (ours)	66.7	**71.7**	**60.1**	**72.9**	44.5	**62.2**
MiniMax M3						
Single model	83.3	80.0	80.2	77.6	61.6	75.2
StructAgent (ours)	**95.5**	**84.4**	**82.9**	**83.7**	**69.7**	**78.9**

instantiate the same state-centered framework in Minecraft as a cross-domain generalization study; environment details, task construction, and additional analysis are provided in Appendix F.

Compared systems. We separate contextual reported results from controlled self-run comparisons. Reported frontier single-model rows are taken from the OSWorld-Verified leaderboard [4]. Reported framework rows use the original results of Agent S3 [12], OS-Symphony [9], and VLAA-GUI [8]. These rows provide reference points under their original settings. Our controlled comparison is the self-run open-backbone block, where StructAgent and reproduced single-model, Agent S3, OS-Symphony, and VLAA-GUI baselines are run under matched model and runner settings.

Controlled protocol. For OSWorld-Verified, all controlled runs use a 100-step budget and the same runner and scoring pipeline. We evaluate Qwen3.5-9B and Qwen3.5-27B from the Qwen vision-language model family [13], and MiniMax-M3 [10]. Unless otherwise stated, main results use a single rollout. Behavior Best-of-N [12] is studied separately as a test-time scaling procedure in Section 5. Prompt schemas in Appendix H, verification registry details in Appendix C, and complete runtime configurations in Appendix G are provided for reproducibility.

4.2 Main Results on OSWorld-Verified

Table 1 reports OSWorld-Verified success rates. The table first lists reported reference results, then compares self-run open-backbone agents under matched runner settings.

The controlled open-backbone results support three findings.

First, StructAgent consistently strengthens the same backbone under matched runner settings. The largest gains appear on weaker open models: Qwen3.5-9B improves from 27.0% to 46.9%, and Qwen3.5-27B improves from 31.6% to 62.2%. On the stronger MiniMax-M3 backbone, StructAgent further improves the single-model baseline and reaches 78.9%, the strongest open-model result in

Table 2: Mind2Web generalization results. Success rate (%) is grouped by website category and graded with the same WebJudge-style evaluator. Rows are grouped by open backbone. Ent. = Entertainment.

System	Info	Service	Ent.	Shopping	Travel	Overall
Qwen3.5-9B						
Agent S3	53.8	**50.6**	27.6	42.1	**35.5**	46.0
OS-Symphony	42.9	49.4	31.0	47.4	32.3	42.6
VLAA-GUI	44.2	**50.6**	**31.0**	**52.6**	**35.5**	44.3
StructAgent (ours)	**61.5**	50.0	24.1	36.4	33.3	**47.3**
Qwen3.5-27B						
Agent S3	61.5	**55.3**	34.5	44.7	**41.9**	51.7
OS-Symphony	49.4	51.9	37.9	52.6	38.7	47.7
VLAA-GUI	51.9	53.2	**44.8**	**57.9**	**41.9**	50.6
StructAgent (ours)	**65.4**	54.9	34.5	39.4	38.1	**52.3**

Table 1. These results suggest that the structured state-centered loop is not tied to a particular model scale.

Second, the gains are largest when task progress can be structured and verified. Office tasks show the clearest improvement, especially on Qwen3.5-9B, where StructAgent substantially outperforms both the single-model baseline and the strongest reproduced framework baseline. This is consistent with the design of StructAgent: spreadsheet and document tasks expose durable intermediate states, captured values, and application-level checks, allowing the verifier-maintained state to reduce brittle reliance on raw GUI history. With Qwen3.5-27B, the advantage becomes broader, with StructAgent leading the 27B group in Office, Daily, Professional, and Overall, suggesting that stronger backbones can better use state, evidence, and recovery feedback.

Third, cross-application tasks remain the hardest setting. StructAgent improves substantially with MiniMax-M3 on Multi-Apps, but the smaller Qwen3.5-9B backbone still trails OS-Symphony on this group. These tasks require not only reliable verification, but also reasoning over dependencies across applications, files, and handoffs. This indicates that state-centered structure helps preserve progress, while complex cross-application planning remains a bottleneck for smaller backbones.

4.3 Web Generalization

Mind2Web [11] tests whether the same state-centered design transfers beyond executable desktop scoring. All self-run systems are evaluated with the same WebJudge-style protocol [14], where the judge receives task points, key screenshots, and action history instead of relying on self-reported completion.

The web results show modest but consistent transfer. With Qwen3.5-9B, StructAgent reaches 47.3% overall, slightly above the strongest reproduced baseline at 46.0%. With Qwen3.5-27B, StructAgent reaches 52.3%, again leading the controlled 27B group. The strongest gains appear in information-seeking tasks, where progress can often be tracked through searched facts, page states, URLs, and visible evidence.

The weaker categories clarify the limits of the current implementation. Entertainment, shopping, and travel tasks often depend on site-specific flows, dynamic widgets, pop-ups, and transactional choices. These cases create action-grounding bottlenecks that are not solved by state tracking alone. Thus, verifier-maintained state helps preserve web progress, but robust browser exploration and interaction with dynamic sites remain open problems.

4.4 Minecraft Generalization

Minecraft is used as a cross-domain instantiation rather than a replacement for desktop CUA evaluation. The goal is to test whether the same state-centered structure can be reused when the evidence source changes. In desktop tasks, verification may rely on screenshots, accessibility trees, files,

URLs, or application-state probes. In Minecraft, the verifier instead uses inventory-based checks: subgoals are represented as $(\text{VERB}, \text{ITEM}, n)$, such as $(\text{CRAFT}, \texttt{iron_pickaxe}, 1)$, and state advances only when the target inventory condition is verified.The same separation is preserved: the planner, driven by GPT-4o, selects subgoals, specialized actors execute navigation, crafting, or placed-block interactions, and the verifier commits progress only after inventory evidence..

Table 3: Minecraft success rate (%) results. We uses GPT-4o as the backbone model; Optimus-1 numbers are published results from [15].

Agent variant	Wooden	Stone	Iron	Golden	Redstone	Avg.
StructAgent (ours)	**100**	70	**65**	**86**	**57**	**76**
Optimus-1	99	**92**	47	9	25	59

The Minecraft results support the abstraction-level claim: StructAgent's state-update discipline is not tied to desktop screenshots. It achieves a 76% task-weighted average success rate, compared with 59% for Optimus-1, and improves success on the Iron, Golden, and Redstone tiers, where long dependency chains make verified subgoal tracking especially useful. Performance is not uniformly better, however: on Stone tasks, Optimus-1 has a higher success rate, and StructAgent is sometimes less step-efficient even when it succeeds. This matches the intended role of the study. It demonstrates that the same structured state and verifier-committed progress mechanism can be instantiated with a different evidence source, while leaving domain-specific actor efficiency as a separate bottleneck.

5 Analysis

We analyze how the unified state and structured workflow change agent behavior through four questions: whether verifier-committed state updates prevent false progress commits, whether structured evidence improves verification, when such evidence is available, and what failures remain after unified-state progress control.

Q1: Does the unified state prevent false progress commits? The unified-state progress control implements StructAgent's core principle: planner and actor completion claims are proposals, not progress updates. In long-horizon tasks, a visually plausible screen may still hide a missing file, an edit to the wrong object, or a later action that overwrites earlier work. StructAgent therefore advances a subgoal only when a verifier-backed decision commits it to the unified state.

Figure 3 illustrates this behavior in a multi-application task. The actor appears to export an image from a document attachment, but the verifier finds no matching file evidence and keeps the image-export subgoal pending. This blocks a false DONE transition and triggers recovery through a structured probe. Once the extracted image is found on disk, the verifier commits the subgoal, allowing the planner to safely complete the wallpaper-setting step.

Q2: Does structured evidence improve verification? We isolate the verifier on 354 scored Qwen3.5-27B OSWorld-Verified trajectories, using the OSWorld environment checker as ground truth. We compare StructAgent's structured verifier with a screenshot-only visual verifier at the final DONE gate.

Figure 4(a) shows that structured verification raises recall from 0.67 to 0.81 and F1 from 0.70 to 0.76, while keeping precision close to the visual-only verifier. This indicates that structured evidence mainly recovers true completions that are difficult to validate from the final screenshot alone, rather than merely accepting more DONE claims.

We further review 100 intermediate cases where the structured and visual verifiers disagree, using the instruction, screenshots, verifier evidence, and probe outputs. As shown in Figure 4(b), structured verification is correct in 63% of disagreements, compared with 31% for the visual-only verifier and 6% unclear. Its advantage comes mainly from latent evidence, such as files, command outputs, URLs,

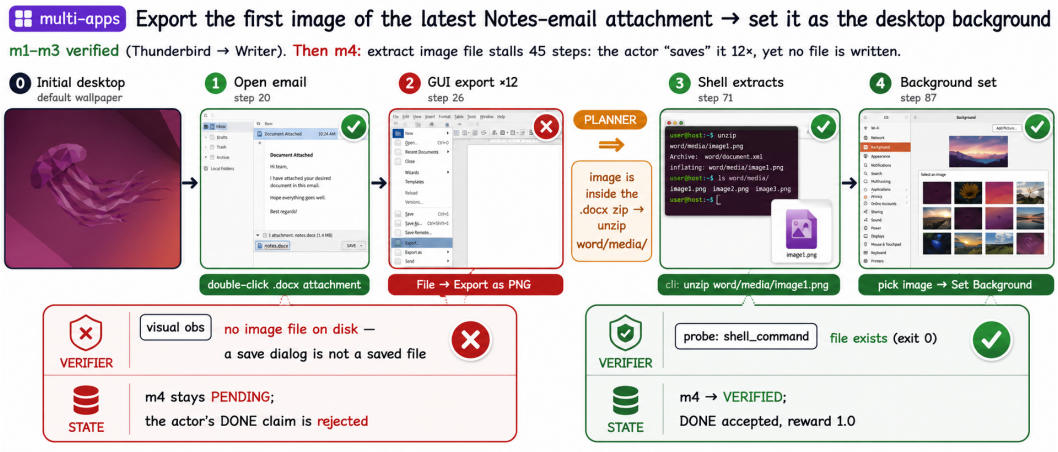
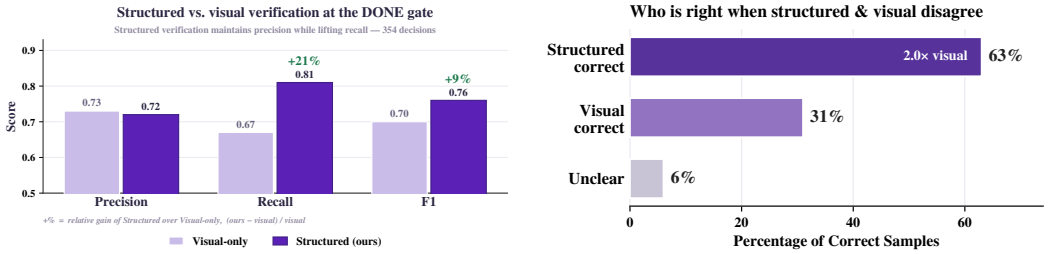


Figure 3: Recovery in a multi-app task. The verifier rejects unsupported GUI-export progress, keeps the subgoal pending, and commits completion only after shell evidence confirms the extracted image.



(a) DONE-gate verification.

(b) Disagreement review.

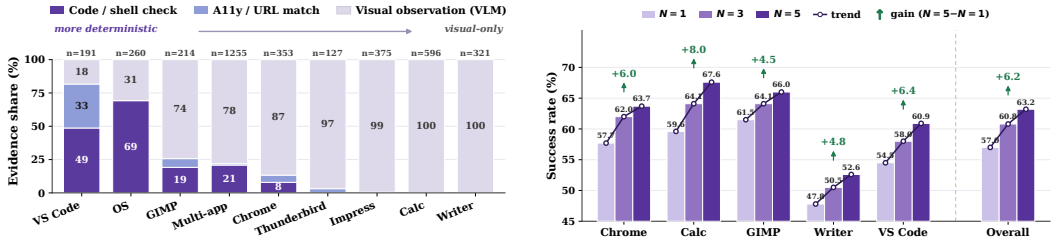
Figure 4: Structured verification improves final completion checking and is more often correct when it disagrees with visual-only verification.

and application state, that is not reliably visible in screenshots. Visual-correct cases expose stale probes or proxy matches, suggesting a hybrid verifier that prioritizes reliable structured checks while retaining visual evidence as a fallback and cross-check. Representative cases appear in Appendix E.2.

Q3: When is structured evidence useful? Figure 5(a) shows that structured verification is most useful when the environment exposes durable task state. VS Code and OS tasks often provide files, shell outputs, URLs, or accessibility checks, allowing the verifier to rely less on visual judgment. In contrast, Calc, Writer, Impress, and several browser or email tasks remain more visual because success conditions are embedded in GUI state or document layout. Thus, StructAgent benefits most when task-relevant evidence is externally checkable; when success is primarily visual, richer app-specific checks and verifier training become the next bottleneck.

We also test whether this evidence discipline helps at test time. Behavior Best-of-N (BBON) [12] is not a core component of StructAgent, but we use it to ask whether structured candidate-level evidence helps a verifier select among completed rollouts. Figure 5(b) shows that moving from one to three rollouts improves every selected domain, and five rollouts further improves the overall score. Most gains appear early, so BBON is best viewed as an optional test-time tradeoff.

The domain pattern mirrors runtime verification. Calc, Chrome, and VS Code benefit more because candidate trajectories expose clearer final state through cells, pages, files, URLs, or code-visible artifacts. Writer and GIMP improve less because success is more often tied to visual layout or image-level details. Structured selection is therefore strongest when candidate evidence aligns with the benchmark outcome, and weaker when the verifier must fall back to visual judgment.



(a) Runtime evidence sources. **(b)** Test-time BBON scaling.

Figure 5: Structured evidence at runtime and test time.

Q4: What failures remain? Finally, we audit 83 score-zero OSWorld-Verified trajectories from the Qwen3.5-27B run, capped at 20 failures per domain. Each trajectory is assigned a primary failing role and failure type using saved screenshots, action traces, verifier logs, and final checker feedback, rather than the agent's own diagnosis. Failure types include state verification errors, execution failures, information errors, strategy errors, long-horizon handoff failures, and external environment blockers.

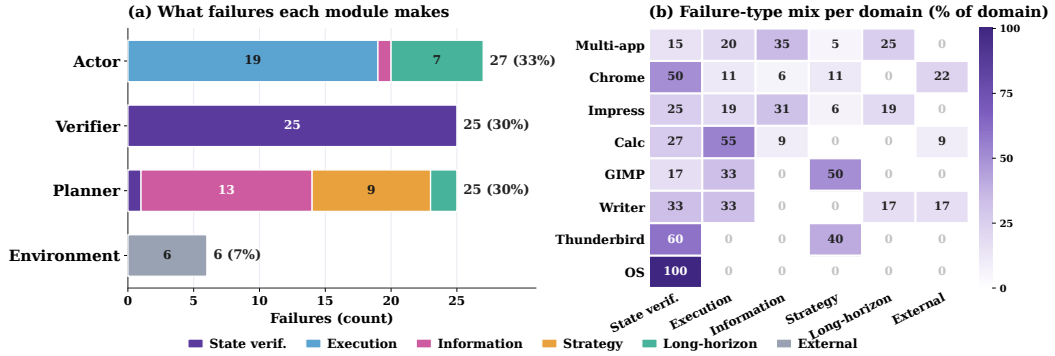


Figure 6: Failure analysis on score-zero OSWorld-Verified trajectories. Left: primary failing role and failure type. Right: domain-level failure-type mix.

Figure 6 shows that remaining errors are not only actor grounding failures. Actor failures account for 33% of audited cases, while planner and verifier failures each account for 30%, and environment-side issues account for 7%. The largest single failure type is state verification: the system sometimes accepts weak evidence, misses a completed condition, or fails to detect that later actions invalidated earlier progress.

The domain breakdown shows that no single module explains all failures. Calc, GIMP, and Writer are more execution-heavy; Multi-app and Impress expose more information-selection and handoff failures; Chrome and Thunderbird show more state-verification or external blockers. This supports the systems view behind StructAgent: long-horizon reliability requires better evidence design, stronger verifier training, and domain-aware execution support, not merely scaling one planner or actor module.

6 Related Work

Long-horizon computer-use agents require more than a capable action model: they need mechanisms for planning over extended tasks, checking whether progress has actually been made, recovering from partial failures, and reusing reliable procedures. We therefore organize related work by these structural roles rather than by individual systems. Table 4 summarizes the main design patterns. Prior work often strengthens one part of the agent loop; StructAgent connects planning, verification, recovery, and tool/skill reuse through a shared verifier-maintained state.

Table 4: Positioning StructAgent among representative structured-agent systems. **Plan**: long-horizon decomposition or orchestration; **Ver.**: explicit progress or outcome verification; **Rec.**: adaptive recovery or replanning; **T/S**: reusable tools, workflows, or skills; **SS**: shared task-progress state; **VC**: verifier-committed state updates. ○ indicates a partial or auxiliary mechanism.

Line of work	Representative works	Plan	Ver.	Rec.	T/S	SS	VC
Long-horizon agent systems	OS-Copilot [6], Cradle [5], Agent S2/S3 [7, 12], OS-Symphony [9], VLAA-GUI [8]	✓	○	✓	○	✗	✗
Progress verification	OpenComputer [16], VAGEN [17], GUI-Shepherd [18], V-Droid [19], VeriGUI [20]	✗	✓	○	✗	✗	✗
Memory, tools, and skills	Agent Workflow Memory [21], SkillWeaver [22], ToolCUA [23], OS-Copilot [6], Voyager [24]	○	✗	○	✓	✗	✗
StructAgent	**Unified verifier-maintained state for planning, acting, recovery, and tool use**	✓	✓	✓	✓	✓	✓

Long-horizon Agent Design. Benchmarks for web and desktop control make digital agents measurable on tasks that require planning, navigation, file editing, cross-application handoffs, and recovery from partial failures [3, 4, 11, 25]. Recent GUI and computer-use models improve perception and action grounding, while system-level agents introduce decomposition, specialist roles, reflection, recovery, tools, or orchestration to make longer tasks more controllable [5–9, 12, 26–30]. This line motivates our comparison setting: long-horizon reliability is an agent-design problem, not only a model-scaling problem. StructAgent follows this systems view, but structures task progress itself as a shared state rather than only structuring roles, prompts, or execution routes.

Verification and Progress Control. A second line studies how to decide whether an agent has actually made progress or completed a task. Reward and verifier benchmarks show that completion judgment is fragile when evidence is incomplete, stale, or mostly visual [31–33]. Process-level and environment-state methods address this issue by checking intermediate actions, hidden application state, or task-specific criteria instead of trusting self-reported DONE claims [8, 16–19, 34–36]. These works are direct comparison cases because StructAgent also treats unsupported progress as unsafe. The difference is that verification is not only a reward, critic, STOP gate, or final judge in StructAgent; it is the mechanism that commits, preserves, or invalidates progress in the shared state.

Memory, Tools, and Skill Reuse. Memory and tool-use methods address another long-horizon bottleneck: agents should not rediscover the same workflows, commands, or interaction patterns from scratch. Prior work stores reusable experience, induces workflows or skills, and builds tool surfaces that make execution more reliable across repeated tasks [6, 21–23, 37–45]. Similar ideas also appear in embodied or game environments, where skill libraries and inventory-level progress provide natural interfaces for long dependency chains [15, 24, 46, 47]. StructAgent also uses memory and tools, but separates reusable advice from current task truth. A retrieved workflow or tool call can help the actor attempt a subgoal, while only evidence-backed verifier events can update the state.

7 Conclusion

We presented **StructAgent**, a state-centered framework for long-horizon computer use based on two complementary designs: a unified state that provides a compact and verifiable representation of task progress, and a structured workflow that regulates progress through verifier-backed state transitions. Rather than treating long-horizon execution as continuous generation over an ever-growing interaction history, StructAgent organizes planning, acting, and verification around an explicit causal progress structure. This design enables transparent progress checkpointing, evidence-driven task completion, targeted failure recovery, and tool-assisted execution while preserving a consistent and auditable working process. Extensive experiments demonstrate that StructAgent consistently improves diverse LLM and VLM backbones on long-horizon computer-use benchmarks, achieving a new open-source state-of-the-art on OSWorld-Verified and generalizing beyond desktop environments to Minecraft.

Looking forward, we believe the proposed state-centered paradigm can naturally extend beyond computer use to broader agentic systems, including web agents, embodied agents, and multi-agent collaboration. Future work may further enrich the state with richer semantic and causal representations, learn adaptive workflow policies, and investigate scalable verification mechanisms for increasingly complex real-world tasks.

References

[1] W. X. Zhao et al. "A Survey of Large Language Models". In: *arXiv preprint arXiv:2303.18223* (2023).

[2] L. Wang et al. "A Survey on Large Language Model based Autonomous Agents". In: *arXiv preprint arXiv:2308.11432* (2023).

[3] T. Xie et al. "OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments". In: *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2404.07972. 2024.

[4] XLANG Lab. *Introducing OSWorld-Verified*. `https://xlang.ai/blog/osworld-verified`. 2025.

[5] W. Tan, W. Zhang, X. Xu, et al. "Cradle: Empowering Foundation Agents Towards General Computer Control". In: *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2403.03186. 2024.

[6] Z. Wu et al. "OS-Copilot: Towards Generalist Computer Agents with Self-Improvement". In: *ICLR 2024 LLM Agents Workshop* (2024). arXiv:2402.07456.

[7] S. Agashe, K. Wong, V. Tu, J. Yang, A. Li, and X. E. Wang. "Agent S2: A Compositional Generalist-Specialist Framework for Computer Use Agents". In: *arXiv preprint arXiv:2504.00906* (2025).

[8] Q. Han et al. "VLAA-GUI: Knowing When to Stop, Recover, and Search, A Modular Framework for GUI Automation". In: *arXiv preprint arXiv:2604.21375* (2026).

[9] B. Yang et al. "OS-Symphony: A Holistic Framework for Robust and Generalist Computer-Using Agent". In: *arXiv preprint arXiv:2601.07779* (2026).

[10] X. Lai et al. "MiniMax Sparse Attention". In: *arXiv preprint arXiv:2606.13392* (2026). Reports the public MiniMax-M3 model release.

[11] X. Deng et al. "Mind2Web: Towards a Generalist Agent for the Web". In: *NeurIPS Datasets and Benchmarks Track*. arXiv:2306.06070. 2023.

[12] G. Gonzalez-Pumariega, V. Tu, C.-L. Lee, J. Yang, A. Li, and X. E. Wang. "Scaling Agents for Computer Use". In: *arXiv preprint arXiv:2510.02250* (2025).

[13] S. Bai, Y. Cai, R. Chen, et al. "Qwen3-VL Technical Report". In: *arXiv preprint arXiv:2511.21631* (2025).

[14] T. Xue et al. "An Illusion of Progress? Assessing the Current State of Web Agents". In: *COLM*. arXiv:2504.01382. 2025.

[15] Z. Li, Y. Xie, R. Shao, G. Chen, D. Jiang, and L. Nie. "Optimus-1: Hybrid Multimodal Memory Empowered Agents Excel in Long-Horizon Tasks". In: *Advances in Neural Information Processing Systems*. arXiv:2408.03615. 2024.

[16] J. Wei et al. "OpenComputer: Verifiable Software Worlds for Computer-Use Agents". In: *arXiv preprint arXiv:2605.19769* (2026).

[17] C. Cui, J. Huang, S. Wang, L. Zheng, Q. Kong, and Z. Zeng. "Agentic Reward Modeling: Verifying GUI Agent via Online Proactive Interaction". In: *arXiv preprint arXiv:2602.00575* (2026).

[18] "GUI-Shepherd: Reliable Process Reward and Verification for Long-Sequence GUI Tasks". In: *arXiv preprint arXiv:2509.23738* (2025).

[19] G. Dai et al. "Advancing Mobile GUI Agents: A Verifier-Driven Approach to Practical Deployment". In: *arXiv preprint arXiv:2503.15937* (2025). Introduces V-Droid.

[20] Y. Zhang, X. Xue, X. Wu, et al. "Don't Act Blindly: Robust GUI Automation via Action-Effect Verification and Self-Correction". In: *arXiv preprint arXiv:2604.05477* (2026).

[21] Z. Wang, J. Mao, D. Fried, and G. Neubig. "Agent Workflow Memory". In: *International Conference on Learning Representations (ICLR)*. arXiv:2409.07429. 2025.

[22] B. Zheng et al. “SkillWeaver: Web Agents can Self-Improve by Discovering and Honing Skills”. In: *arXiv preprint arXiv:2504.07079* (2025).

[23] X. Hu et al. “ToolCUA: Towards Optimal GUI-Tool Path Orchestration for Computer Use Agents”. In: *arXiv preprint arXiv:2605.12481* (2026).

[24] G. Wang et al. “Voyager: An Open-Ended Embodied Agent with Large Language Models”. In: *NeurIPS 2023 FMDM Workshop*. arXiv:2305.16291. 2023.

[25] R. Bonatti, D. Zhao, F. Bonacci, D. Dupont, S. Abdali, et al. “Windows Agent Arena: Evaluating Multi-Modal OS Agents at Scale”. In: *arXiv preprint arXiv:2409.08264* (2024).

[26] Y. Qin, Y. Ye, J. Fang, H. Wang, S. Liang, et al. “UI-TARS: Pioneering Automated GUI Interaction with Native Agents”. In: *arXiv preprint arXiv:2501.12326* (2025).

[27] Y. Xu et al. “Aguvis: Unified Pure Vision Agents for Autonomous GUI Interaction”. In: *ICML*. arXiv:2412.04454. 2025.

[28] Z. Wu et al. “OS-Atlas: A Foundation Action Model for Generalist GUI Agents”. In: *arXiv:2410.23218* (2024).

[29] X. Wang, B. Wang, D. Lu, J. Yang, T. Xie, et al. “OpenCUA: Open Foundations for Computer-Use Agents”. In: *arXiv preprint arXiv:2508.09123* (2025).

[30] Z. Liu, J. Xie, Z. Ding, Z. Li, B. Yang, et al. “ScaleCUA: Scaling Open-Source Computer Use Agents with Cross-Platform Data”. In: *arXiv preprint arXiv:2509.15221* (2025).

[31] X. H. Lù, A. Kazemnejad, N. Meade, A. Patel, et al. “AgentRewardBench: Evaluating Automatic Evaluations of Web Agent Trajectories”. In: *Conference on Language Modeling (COLM)*. arXiv:2504.08942. 2025.

[32] H. Lin, X. Tan, Y. Qin, et al. “CUARewardBench: A Benchmark for Evaluating Reward Models on Computer-using Agent”. In: *arXiv preprint arXiv:2510.18596* (2025).

[33] M. Sumyk and O. Kosovan. ““Are We Done Yet?”: A Vision-Based Judge for Autonomous Task Completion of Computer Use Agents”. In: *arXiv preprint arXiv:2511.20067* (2025).

[34] T. Xiong, X. Hu, Y. Chen, et al. “GUI-PRA: Process Reward Agent for GUI Tasks”. In: *arXiv preprint arXiv:2509.23263* (2025).

[35] H. Chae, S. Kim, et al. “Web-Shepherd: Advancing PRMs for Reinforcing Web Agents”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2505.15277. 2025.

[36] C. Rosset, P. Sharma, A. Zhao, M. Gonzalez-Fernandez, and A. Awadallah. “The Art of Building Verifiers for Computer Use Agents”. In: *arXiv preprint arXiv:2604.06240* (2026).

[37] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang. “A-Mem: Agentic Memory for LLM Agents”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2502.12110. 2025.

[38] Y. Liu et al. “Contextual Experience Replay for Self-Improvement of Language Agents”. In: *Proceedings of ACL*. arXiv:2506.06698. 2025.

[39] Z. Sun et al. “SEAgent: Self-Evolving Computer Use Agent with Autonomous Learning from Experience”. In: *arXiv preprint arXiv:2508.04700* (2025).

[40] F. Mi et al. “Darwinian Memory: A Training-Free Self-Regulating Memory System for GUI Agent Evolution”. In: *arXiv preprint arXiv:2601.22528* (2026).

[41] “EchoTrail-GUI: Building Actionable Memory for GUI Agents via Critic-Guided Self-Exploration”. In: *arXiv preprint arXiv:2512.19396* (2025).

[42] L. Sun, J. Zhang, S. Wang, and Z. Wei. “MAGNET: Towards Adaptive GUI Agents with Memory-Driven Knowledge Evolution”. In: *arXiv preprint arXiv:2601.19199* (2026).

[43] W. Cheng et al. “MGA: Memory-Driven GUI Agent for Observation-Centric Interaction”. In: *arXiv preprint arXiv:2510.24168* (2025).

[44] T. Q. Nguyen, S. Desai, R. H. Anwar, F. Shaik, V. Suryanarayanan, and V. Chowdhary. “VerificAgent: Domain-Specific Memory Verification for Scalable Oversight of Aligned Computer-Use Agents”. In: *arXiv preprint arXiv:2506.02539* (2025).

[45] S. Zhu, W. Wu, K. Zhou, S. Wang, and B. Huang. “Hybrid Self-evolving Structured Memory for GUI Agents”. In: *arXiv preprint arXiv:2603.10291* (2026).

[46] S. Lifshitz, K. Paster, H. Chan, J. Ba, and S. McIlraith. “STEVE-1: A Generative Model for Text-to-Behavior in Minecraft”. In: *arXiv preprint arXiv:2306.00937* (2023).

[47] Z. Wang et al. “JARVIS-1: Open-World Multi-task Agents with Memory-Augmented Multimodal Language Models”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2023). arXiv:2311.05997.

[48] B. Wang, D. Lu, J. Wang, T. Bai, S. Liu, Z. Zhang, et al. "CUA-Gym: Scaling Verifiable Training Environments and Tasks for Computer-Use Agents". In: *arXiv preprint arXiv:2605.25624* (2026).

[49] N. Reimers and I. Gurevych. "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks". In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*. arXiv:1908.10084. 2019.

[50] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou. "MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers". In: *Advances in Neural Information Processing Systems*. arXiv:2002.10957. 2020.

[51] J. Johnson, M. Douze, and H. Jégou. "Billion-Scale Similarity Search with GPUs". In: *IEEE Transactions on Big Data* 7.3 (2019). arXiv:1702.08734, pp. 535–547.

[52] W. H. Guss et al. "MineRL: A Large-Scale Dataset of Minecraft Demonstrations". In: *arXiv preprint arXiv:1907.13440* (2019).

[53] Z. Dai, Z. Yang, Y. Yang, J. Carbonell, Q. V. Le, and R. Salakhutdinov. "Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context". In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. arXiv:1901.02860. 2019.

[54] J. Park, J. Cho, and S. Ahn. "MrSteve: Instruction-Following Agents in Minecraft with What-Where-When Memory". In: *International Conference on Learning Representations*. arXiv:2411.06736. 2025.

A Additional OSWorld Results and Ablations

Table 5 reports the full 10-domain OSWorld breakdown for controlled self-run rows. Scores are computed from the summed task scores divided by the number of scored tasks in each domain, so partial credit is preserved when the benchmark scorer returns it.

Table 5: OSWorld-Verified per-domain self-run results (success rate %). Chr. = Chrome; Th. = Thunderbird; VS = VS Code.

System	Backbone	Chr.	GIMP	Calc	Impress	Writer	Multi	OS	Th.	VLC	VS	Overall
StructAgent (ours)	Qwen3.5-9B	57.7	61.5	59.6	44.5	47.8	27.5	62.5	60.0	28.9	54.5	46.9
Single model	Qwen3.5-9B	41.2	26.9	8.5	27.5	30.4	17.7	41.7	60.0	23.5	36.4	27.0
Agent S3	Qwen3.5-9B	60.8	50.0	29.8	27.5	34.8	26.6	54.2	71.4	41.3	72.7	40.8
OS-Symphony	Qwen3.5-9B	52.1	65.4	32.6	25.4	50.0	32.6	56.5	71.4	52.9	50.0	43.3
VLAA-GUI	Qwen3.5-9B	54.3	69.2	21.3	20.1	56.5	28.5	69.6	46.7	30.7	63.6	40.2
StructAgent (ours)	Qwen3.5-27B	60.8	76.9	76.6	65.8	73.9	44.5	66.7	66.7	51.6	68.2	62.2
Single model	Qwen3.5-27B	47.7	30.8	15.2	32.6	34.8	18.3	50.0	60.0	28.9	45.5	31.6
Agent S3	Qwen3.5-27B	54.3	61.5	72.3	42.3	73.9	54.0	70.8	66.7	67.5	72.7	60.2
OS-Symphony	Qwen3.5-27B	52.1	53.8	48.9	36.2	69.6	43.5	70.8	80.0	56.2	77.3	52.8
VLAA-GUI	Qwen3.5-27B	45.6	46.2	31.9	16.8	56.5	47.3	83.3	80.0	63.5	81.8	48.2
Single model	MiniMax-M3	76.0	76.9	80.9	73.7	91.2	61.6	83.3	86.7	86.6	78.3	75.2
StructAgent (ours)	MiniMax-M3	73.8	84.6	89.1	74.3	95.6	69.7	95.5	93.3	98.2	82.6	78.9

B Web Benchmark Details

Benchmarks. We prepare a Mind2Web [11] web-generalization evaluation. Results are grouped by website category: information, service, entertainment, shopping, and travel.

Mind2Web evaluation. For Mind2Web, we follow the evaluation protocol and prompt design of WebJudge [14]. The judge receives key task points, key screenshots, and the action history, and decides whether the trajectory satisfies the user instruction. We use Qwen3.5-27B [13] as the judge model for the Mind2Web rows. This keeps the comparison under a shared evaluation protocol rather than relying on framework-specific success declarations.

Table 6: Reliability of the Mind2Web judge used in our evaluation. Agreement is measured against human labels on a held-out validation set.

Judge	N	Agreement (%)	SR Gap (pp)	False Pos. (%)	False Neg. (%)
Qwen3.5-27B WebJudge-style evaluator	150	86.0	4.7	4.7	9.3

On this validation set, the judge agrees with human labels on 129 of 150 examples, with 7 false positives and 14 false negatives. This corresponds to a 4.7 point success-rate gap against human labels. The lower false-positive rate indicates a conservative evaluator, which is preferable for Mind2Web because over-crediting failed trajectories would inflate the reported success rate.

Reported and self-run rows. Self-run rows use the same evaluation code and judge model within each benchmark. Published reference rows, when included, follow the original benchmark settings and are shown only as external reference points.

C State, Verification Registry, and Execution Surfaces

State schema. The implementation stores task progress in a typed `VerificationRecord`. The main milestone object is `Outcome`. Each outcome contains an id, description, evidence hint, dependencies, an optional app tag for multi-app tasks, an optional `VerifySpec`, and an authoritative state from `pending`, `verified`, and `reverted`, where `reverted` is the implementation counterpart of INVALIDATED. Planner and actor outputs are recorded in the timeline, but they do not directly change an outcome state.

Table 7: Implementation-level state components.

Object	Main fields	Updated by	Purpose
`VerificationRecord`	`initial_context`, `required_outcomes`, legacy `done`, `failed_paths`, `completed_strategies`, `slots`, and `global_facts`.	Initializer, verifier, planner loop	Provides the shared task state rendered back to the planner.
`Outcome`	`id`, `description`, `evidence_hint`, `depends_on`, `verify`, `app`, `state`, rejection counters, facts, evidence, and failures.	Initializer and verifier	Represents one durable gating milestone.
`VerifySpec`	One structured verifier specification attached to an outcome, such as `file_grep` or `calc_verify`.	Initializer and verifier rewrite logic	Defines how an outcome can be checked without free-form code generation.
`StepRecord` and `TimelineEvent`	Screenshot, a11y text, URL, actor response, emitted action code, planner decision, key nodes, and perceiver snapshot.	Runtime loop	Preserves the execution history used by planner rendering, verifier context, debugging, and audit.
`Evidence`, `Fact`, and `Failure`	Verifier traces, bound slot values, extracted facts, and failed strategy or action records.	Verifier, extraction handlers, failure attribution	Carries grounded useful values and recovery evidences across later turns.

Table 8: Actual `VerifySpec` kinds used by the verification registry.

Kind	Required fields and evidence	Typical use
`file_grep`	`file_path`, regex `patterns`, optional `all_must_match` and `proximity_lines`; reads a VM file and matches text.	Plaintext files, saved configs, bookmarks, and exported text artifacts.
`url_match`	`url_pattern`; extracts the active browser URL from address-bar a11y rows.	Chrome navigation and page-open conditions.
`a11y_match`	Optional `tag`, `text_contains`, `state_contains`, and visual clauses.	Visible UI state, controls, dialogs, forms, Thunderbird, VLC, and other live app states.
`calc_verify`	A list of `calc_checks` whose ops are validated against the Calc action library.	Spreadsheet cells, formulas, formats, sorting, frozen panes, and related Calc state.
`writer_verify`	A list of `writer_checks` validated by the Writer action library.	Document text, paragraph properties, and character formatting.
`impress_verify`	A list of `impress_checks` validated by the Impress action library.	Slide text, shapes, notes, and presentation formatting.
`shell_command`	An argv list plus expected or forbidden substrings. Boundary probes are guarded by a read-only allowlist such as `which`, `pgrep`, `test`, `gsettings get`, `dpkg -l`, and `snap list`.	File existence, process state, installed packages, and system settings.

Actor-loop and boundary verification. In the current boundary-verification mode, per-step KeyNode polling is bypassed. The runtime still records each `StepRecord`, but milestone verification runs when the actor loop returns control to the planner, and once more when the planner claims `DONE`. The boundary verifier receives the current screenshot, up to four prior screenshots from the timeline, prefiltered a11y text, the last eight actor responses, the current subgoal, the target outcomes, and optional verifier-memory check recipes. If a target is verified, the corresponding `Outcome` is marked `verified` and stores a `last_verifier_trace` with source, method, reason, evidence, confidence, step index, and the authored check when one was used. Unverified or uncertain outcomes remain in their previous state and are reported back to the planner.

The low-level actor verifier is separate from milestone verification. It does not update the milestone state or verified evidences. It compares the before and after screenshots for the current actor turn, with up to three prior actor-turn screenshots and the last three rendered action-history lines. Its statuses are `complete`, `continue`, `no_effect`, `off_track`, and `stuck`; these statuses decide whether actor control continues or returns to the planner.

Verification registry. The boundary verifier can either judge from observation or request one validated `VerifySpec`. The actual probe kinds are the seven kinds below. They return `True`, `False`, or `None`; a determinate result overrides the observation judgment, while `None` falls back to observation.

Table 9: Execution surfaces and their verification paths.

Surface	Actual actor actions	Verification path
GUI / pyautogui	`click`, `left_double`, `right_single`, `drag`, `type`, `hotkey`, `scroll`, and `wait`. Coordinates are emitted inline on a 0–1000 grid and converted to pixels.	Low-level VLM verifier, screenshots, a11y, and later boundary probes.
Browser direct action	`navigate(url, new_tab)` through the Chrome DevTools interface, plus `finished(answer)` as a text-answer terminal sentinel. Page interaction still uses GUI actions.	`url_match`, `a11y_match`, screenshots, and task-specific final judging.
Host or VM handlers	`cli_run`, `edit_json`, `extract_info`, and `open_app`. These are actor-side tools and can change task state when used by the actor.	Boundary verifier checks the resulting file, app, browser, or verification-record state; verifier-side `shell_command` remains a separate read-only probe.
Structured Office actions	The allowlist includes validated `calc_*`, `writer_*`, and `impress_*` actions, lowered by their action libraries to runtime scripts.	`calc_verify`, `writer_verify`, `impress_verify`, file checks, and visual fallback.
Terminal actor claims	`impossible` maps to an `IMPOSSIBLE` sentinel; planner-level `DONE` is accepted only after the verification gate and DONE auditor.	Feasibility logic, boundary verification on pending leaves, and the DONE auditor.

Registry selection and fallback. The prompt exposes only the probe kinds allowed by the current domain's trust map. For example, Chrome exposes `a11y_match`, `url_match`, and `file_grep`; Calc exposes `calc_verify` and `file_grep`; Writer and Impress expose their corresponding UNO-backed checks plus `file_grep`; OS tasks expose `file_grep` and `shell_command`. The verifier may return a `probe` object, but `spec_from_probe` accepts it only if the kind is allowed, the fields are whitelisted, the spec is valid, and any shell command passes the read-only guard. Invalid probes are ignored. If a valid probe returns `True` or `False`, the method is recorded as `probe:<kind>`; if it returns `None`, the method is recorded as `probe-inconclusive->obs`.

Execution surfaces. The actor emits actions from the decomposer allowlist. These actions may change the environment, but they do not mark progress unless boundary verification later updates an outcome.

Safety, logging, and reproducibility. Verifier-side shell probes are read-only guarded and cannot use shell pipelines, redirects, or free-form interpreters. Actor-side commands such as `cli_run` are separate execution actions and are logged through the normal action history. Boundary-verifier debug files record allowed probes, target outcomes, verdict methods, reasons, evidence, and authored checks; the timeline retains the screenshots, a11y text, actor responses, and action code needed to audit those decisions. Full prompts and structured output schemas are provided in Appendix H.

D Structured Memory and Tool Construction

Memory as reusable advice, not task truth. StructAgent separates memory from state. Memory stores reusable experience from past trajectories: plans that worked, common action patterns, app-specific rules, pitfalls, and verifier check recipes. The state stores what is currently true in the running task. A memory entry can suggest what to try or how to check a milestone, but it cannot mark an outcome as complete. Outcome state is updated only by verifier-backed decisions, as described in Appendix C.

Offline construction from trajectories. The memory bank is built offline from three trajectory sources. The internal source is produced by running our agent on CUA-Gym [48] tasks. These rollouts are launched by `scripts/run_rollout_subbatch.sh`, adapted into the same execution runner, and mined into `results/planner_experience/<domain>/<task_id>.json`. The external sources are normalized AgentNet desktop trajectories from OpenCUA [29], stored under `results/agentnet_normalized`, and normalized Multimodal-Mind2Web [11] web trajectories, stored under `results/mind2web_normalized`. The loader converts all three sources into the same `UnifiedRecord` schema with instruction, domain, subgoals, key actions, outcomes, and pitfalls.

Table 10: Structured memory surfaces and where they enter the runtime loop.

Runtime point	Memory source	Retrieval key	Runtime use
Task start	L2 plan templates	Task instruction through `when_to_use`.	Similar full plans, typical subgoals, and task-level pitfalls for the initial planner.
Task start	L3a cluster summaries	Task instruction through `cluster_summary`.	Broad task-class patterns shared across related tasks.
Task start	L3c domain rules	Direct domain lookup.	Stable app facts, common widgets, shortcuts, and domain-level pitfalls.
Subgoal transition	L1 typical actions	Current subgoal and domain through `subgoal_pattern`.	Concrete action recipes and tools/widgets seen for similar subgoals.
Subgoal transition	L3a individual rules	Current subgoal through `rule` and `applies_when`.	Local gotchas and reusable constraints for the next action.
Boundary verification	Check recipes	Subgoal, target milestone text, and domain through `when_to_use`.	Guidance for probe choice, strict pass criteria, and known verifier traps.

The unified pool is clustered at task and subgoal levels and then polished into several reusable surfaces. Task-level clusters become plan templates; subgoal segments become typical-action memories; loose clusters become general rules and pitfalls; domain summaries become app-level rule sheets. A leakage-cleaning pass abstracts task-instance values such as file names, URLs, product names, and exact answers while preserving reusable procedures. The resulting entries are indexed with `sentence-transformers/all-MiniLM-L6-v2`, based on Sentence-BERT [49] and MiniLM [50], and FAISS [51]. Verifier-side check recipes are built separately from successful verifier intent recipes and indexed by their `when_to_use` field.

Task-level planning memory. At task start, the planner retrieves L2, L3a, and L3c memory. L2 plan templates provide concrete subgoal sequences from similar successful tasks. L3a cluster summaries provide broader patterns and shared pitfalls for the task class. L3c domain rules are loaded by domain rather than by semantic search; they capture stable facts about an application, such as common controls, shortcuts, and failure modes. These entries are rendered as planning advice, not as required outcomes.

Subgoal-level action memory. During subgoal transitions, the actor-side prompt receives L1 typical actions and L3a individual rules. L1 entries are mined from subgoal segments and contain typical actions, approaches, tools or widgets seen, and common pitfalls. L3a rules give broader guidance that still applies locally, such as avoiding a known brittle path or preferring a reliable app-native control. These memories help the actor choose a concrete execution path, but the effect still has to be checked by low-level and boundary verification.

Verifier-side check recipes. Verifier memory is separate from planner and actor memory. The offline builder distills old verifier intent recipes into boundary check recipes that describe which probe kind is reliable for a milestone class, what a strict pass condition should require, and which signals are misleading. At runtime, `retrieve_check_recipes` returns the top recipes for the current subgoal and target milestones. The retrieved recipes are guidance only: the boundary verifier must still author a valid `VerifySpec`, and that spec must pass the registry validation and read-only guards before it can run.

Trajectory-derived tools and typed execution surfaces. StructAgent follows the tool-CUA idea of extracting reusable procedures from past trajectories [23], but the current runtime does not rely on online synthesis of new executable tools. Reusable procedures appear as structured memory entries, and execution happens through typed action surfaces. The active actor-side surfaces include GUI actions, `navigate`, `cli_run`, `edit_json`, `extract_info`, `open_app`, and validated `calc_*`, `writer_*`, and `impress_*` actions. These surfaces reduce brittle low-level GUI operation, but they still only produce actions; verifier events decide whether the task state has advanced.

Safety and logging. Memory retrieval is optional and failure-safe: missing indexes or retrieval errors yield an empty memory block rather than changing the control flow. The runtime dumps rendered memory blocks and retrieval metadata under the results directory for post-hoc inspection. Leakage cleaning and structured rendering keep memory focused on reusable procedures instead of task-specific answers, and no memory entry can directly accept a DONE claim.

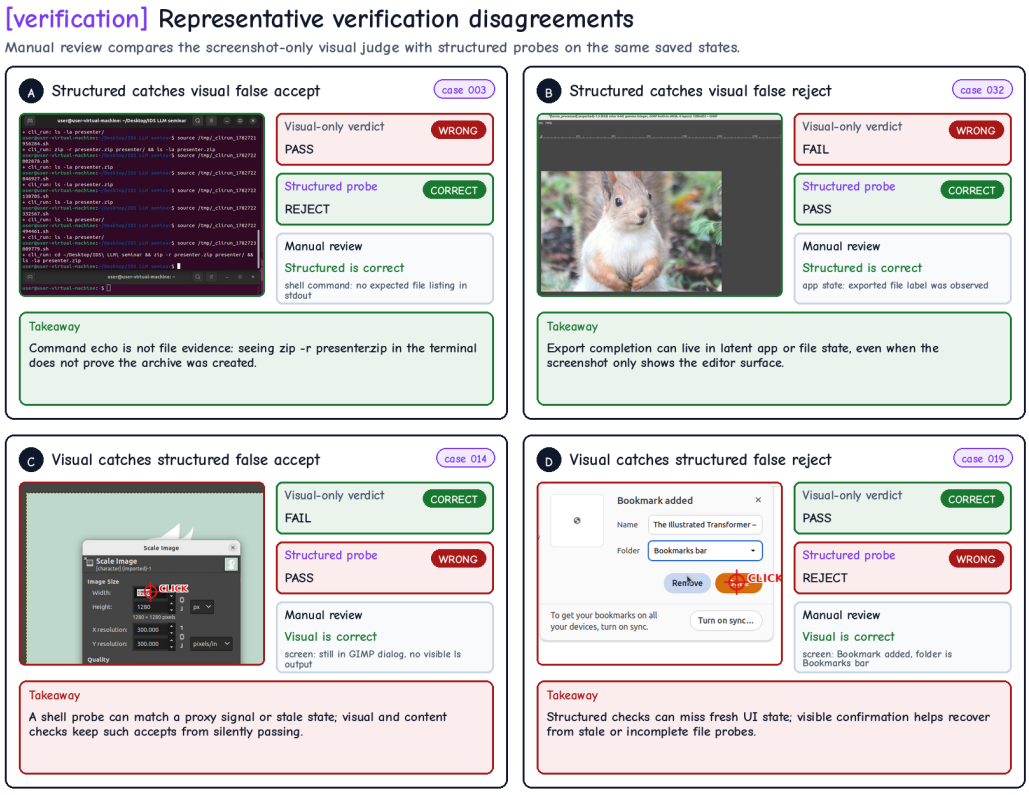


Figure 7: Representative verification disagreements. Structured probes recover latent file or app evidence missed by screenshots, while visual evidence catches stale or proxy probe matches.

E Additional Analysis and Case Studies

E.1 DONE Auditing Prevents Premature Completion

Figure 8 shows a multi-application spreadsheet task where the agent must fill the meeting city for each ICLR, ICML, and NeurIPS conference from 2013 to 2019. In StructAgent, a DONE claim is only a proposal. The DONE auditor rechecks the required city cells, rejects an early 8/21 completion claim, keeps the task open, and accepts the task only after all 21 cities are present. This case illustrates why completion auditing is useful for long table-filling tasks: local actions can look reasonable while the global task state is still incomplete. Without this gate, the baseline performs some correct web lookups, but no module checks the final spreadsheet as a whole: one cell contains a copied search query, many city cells remain blank, and the run exhausts its step budget.

E.2 Representative Verification Disagreements

Figure 7 shows four manually reviewed cases where structured and visual verification disagree. Structured probes catch visual false accepts and false rejects when the relevant evidence lives in

terminal output, file state, or application state. Visual evidence remains useful when a probe matches a proxy signal or misses a fresh UI confirmation.

E.3 Additional BBON Details

We use Behavior Best-of-N [12] as a test-time, verifier-side analysis, not as part of the agent core. For each task we take $N{=}5$ completed rollouts and render each as a behavior narrative: turning-point screenshots, the per-step accessibility state (URL, active tab, element states) paired with each frame, and the final structured probe (for spreadsheets: cell values, formulas, and charts; for Chrome: page and DOM state). A single judge (Qwen3.5-27B [13], structured per-candidate prompt, temperature 0) then selects the most credible trajectory. The judge never sees the ground-truth score; we evaluate its pick afterwards. The main finding is that judge quality is bounded by *evidence*, not reasoning: feeding checker-aligned evidence, such as structured final state and per-step accessibility text, is what turns BBON from harmful or near-random into a positive gain.

F Minecraft Generalization

F.1 Environment

We use the MineRL simulator [52] with Minecraft 1.16.5, the same setting as Optimus-1 [15]. The agent receives a 128×128 RGB observation and emits a 23-element discrete-plus-continuous action dictionary at a fixed 20 Hz tick rate. Episodes terminate on milestone satisfaction or on a configurable step budget (10–15 minutes wall time, equivalent to 12,000–18,000 environment steps). All planning and State-decomposition calls use GPT-4o, while the actors (STEVE-1 and the JARVIS-1 craft/smelt helpers) are separate specialized models, so the reasoning model is held fixed and only the evidence source changes across domains.

F.2 Task suite

Following the Optimus-1 task setting [15], tasks are grouped into seven tiers ordered by canonical crafting depth: Wooden (12 tasks), Stone (10), Iron (17), Golden (7), Redstone (7), Diamond (7), Armor (13). Tier depth corresponds to plan length: a Wooden task is reachable in ≤ 5 sub-tasks under empty inventory; an Iron task requires ≈ 11 sub-tasks; Golden and Redstone reach ≈ 13 sub-tasks. We evaluated Wooden through Redstone.

F.3 Memory bank

The Minecraft memory bank is a recipe-grounded knowledge graph that supplies the planner with canonical sub-task chains for any target item. It is built once offline from three sources: (i) the Minecraft 1.16.5 client recipe data, giving authoritative ingredient slots and counts; (ii) the JARVIS-1 [47] sub-task ontology for non-recipe actions (`mine`, `smelt`, `kill`); (iii) verified successful trajectories from the Optimus-1 experience pool [15], used to align canonical chains with the benchmark.

The bank is a hierarchical directed acyclic graph with 613 item nodes and 848 typed edges; nodes are items, edges encode `produces` with ingredient multiplicity. Each node carries a tier attribute $\in \{\text{WOOD}, \text{STONE}, \text{IRON}, \text{GOLD}, \text{REDSTONE}, \text{DIAMOND}, \text{ARMOR}\}$ from the shortest dependency path. At construction time, every craftable item is compiled to a canonical plan via topologically ordered DFS to mining-source leaves, materialized as a list of $(\text{VERB}, \text{ITEM}, n)$ tuples matching the state milestone format. At plan time the planner queries by target name; the retriever returns the canonical plan filtered by plan-prune, dropping prefix steps already satisfied by the agent's inventory. This is the `memory_bank` component of combined retrieval (§F.5); Optimus-1's AMEP module [15] contributes visual context as a separate joint signal. Retrieval is a target-name dictionary lookup followed by the prune walk, with no LLM call and no embedding model. The bank is serialized as a single JSON loaded once at startup and is read-only during sweeps; failed plans flow into AMEP, not back into the bank.

F.4 Module mapping to desktop CUA

Table 11 shows the per-component correspondence between the desktop and Minecraft instantiations.

Table 11: Per-component correspondence between desktop and Minecraft instantiations of the StructAgent framework. Minecraft actors and memory reuse STEVE-1 [46], JARVIS-1 [47], and Optimus-1 AMEP [15].

Component	Desktop CUA realization	Minecraft realization
State (milestones)	Instruction-derived requirement list.	GPT-4o sub-task list of $(\text{VERB}, \text{ITEM}, n)$ tuples.
Verifier (primary)	Multi-stage screen / DOM / shell-output verifier.	`task_checker` (absolute inventory count) + `SuccessMonitor` (sub-task) + `VerificationRecord` (cumulative goal). Advances when the target inventory condition is verified.
Failure attribution	Routed intervention based on which verifier signal fired.	AMEP failure metadata: failed sub-task, pending outcomes, step count. Plan written to `plan/failed/`.
Actor (action emission)	GUI actor: emits action + coordinates in one call.	STEVE-1 (visuomotor) + JARVIS-1 craft helper + smelt helper, sharing one dispatch interface.
Memory (planning)	Past trajectories indexed by task and DOM context.	AMEP indexed by task, environment, and initial inventory.
Memory (verification)	Verifier examples and failure cases.	Reflection memory + `memory_bank` (recipe-derived canonical plans from a 613-item / 848-recipe knowledge graph).
Replanning	Triggered when verifier rejects observed state.	Triggered on missing-material detection inside craft helper. Plan-prune applied once after retrieval against observed inventory.

F.5 Memory ablation: combined vs. sequential retrieval

The Optimus-1 codebase [15] uses sequential retrieval in `retrieve_plan`: AMEP is consulted first, and `memory_bank` is consulted only on AMEP miss. In a fresh-deployment regime, before tier-specific trajectories have accumulated in AMEP, this systematically returns a near-tier neighbor. With seven Stone-tier successes in AMEP and no Iron-tier successes, every Iron-tier query under sequential retrieval receives a Stone-tier nearest neighbor; the canonical Iron plan present in `memory_bank` is never consulted. The planner extrapolates the Stone-tier example onto the Iron-tier task and truncates the plan at stone-tier items. We observe this deterministically: on Iron task 2 under sequential retrieval, the LLM produces a plan ending in `craft wooden_axe` at $\approx 1{,}820$ steps.

Our combined retrieval inverts the policy: `memory_bank` is the authoritative source of plan structure, AMEP supplements with the most relevant recent visual context, and both are injected into the planner prompt jointly. The same iron task under combined retrieval generates the canonical 11-step plan and the agent reaches the target. On the same eight-fix substrate, sequential retrieval produces $0/17$ Iron-tier passes where combined retrieval produces $11/17$. Beyond pass rate, plan length is qualitatively different: sequential retrieval yields 7–8 sub-task plans truncated at wood or stone tier, combined retrieval yields the full 11-step canonical chain.

F.6 Failure modes

Five distinct failure modes account for all observed Minecraft sweep failures.

The dominant failure on Iron and above is the multi-cell craft retry storm: the agent reaches the final craft step at or near the budget cap, with SUCCESSMONITOR = 1 on every prior sub-task but

Table 12: Failure modes observed across the Wooden–Redstone sweeps.

Failure mode	Signature	Root cause
Multi-cell craft retry storm	100+ verify-retry log lines on a single craft step; budget exhausted with target absent from inventory.	JARVIS-1 craft helper is GUI-blind: cursor moves computed via camera-angle pixel math; sub-pixel drift makes placement clicks miss cells.
Placed-block loss	Agent crafts and opens a `crafting_table`, walks away during STEVE-1 inference; the placed table is no longer reachable.	`return_crafting_table()` exists upstream but its assertion is commented out; silently fails when the policy moves the agent.
Planner truncation	LLM-generated plan ends at a wood- or stone-tier item when the target is iron-tier. Deterministic on affected tasks.	Without iron-tier trajectories in AMEP, sequential retrieval returns a stone-tier nearest neighbor. Mitigated by combined retrieval (§F.5).
Ingredient over-ask	Plan-prune drops every step except mining cobblestone; preload supplies 10, plan asks for 11.	`memory_bank` canonical chains use worst-case ingredient counts; the LLM does not normalize to recipe minimum.
STEVE-1 spatial blindness	Explore-to-find reflection fires; thousands of steps with no resource gain.	STEVE-1 [46] uses Transformer-XL [53] with $\approx$ 6 s frame memory; text conditioning decays; similar spatial-memory limits are discussed in MrSteve [54].

the target item absent from the final inventory. This is orthogonal to the framework contribution; replacing the GUI-blind craft helper with a pixel-aware GUI verifier or a direct-API actor would resolve the failure.

F.7 Case study: `wooden_pickaxe` end-to-end

Figure 9 contrasts our agent's execution of `wooden_pickaxe` against the published Optimus-1 trajectory for the same task (Li et al. [15], Fig. 9). Under our combined retrieval, `memory_bank_v3` supplies the canonical 5-step recipe chain `logs` $\rightarrow$ `planks` $\rightarrow$ `sticks` $\rightarrow$ `crafting_table` $\rightarrow$ `wooden_pickaxe` up front, and the agent completes all five sub-tasks in 2,431 environment steps with no replan. The Optimus-1 baseline reaches the same first four sub-tasks but proposes an under-resourced plan; the `wooden_pickaxe` craft fails at step 5 because the planner does not account for the planks consumed by the prior `crafting_table` step, and the agent must trigger an in-flight replan (`mine 1 log`, `craft 4 planks`, `craft wooden_pickaxe`) to recover.

The trace illustrates three properties relevant to the framework claim. First, the state drives execution: each sub-task is dispatched, verified by inventory delta, and only then marked complete. Second, the multi-stage verifier catches the agent's actual progress: per-cell verify-retry recovers from cursor misses inside the craft helper, SUCCESSMONITOR confirms inventory delta per sub-task, and the `VerificationRecord` declares completion only when the target appears. Third, role separation is preserved across the visuomotor, inventory-slot, and placed-block actors. The replan loop visible in the baseline's bottom-right cell is the observable signature of *retrieval policy* as a design choice: sequential retrieval surfaces a near-tier example whose ingredient counts the LLM extrapolates inaccurately, while combined retrieval surfaces the recipe-grounded canonical chain.

G Reproducibility

Environment and model services. All desktop experiments use the OSWorld execution environment described in Section 4.1, with screenshot observations, a fixed 1920$\times$1080 display, and a 100-step interaction budget. Model services are specified through runtime aliases for the planner/actor,

Table 13: Reproducibility artifacts saved by the runner.

Artifact	Location	Purpose
Runtime configuration	`args.json`	Records model aliases, task split, screen size, step budget, memory settings, and verifier/auditor flags.
Task outcome	`result.txt` and retry summary files	Stores final success or failure status for aggregation.
Verification trace	`boundary_verify/`	Stores milestone verification decisions, evidence, and rejected or confirmed state updates.
DONE audit trace	`done_auditor/`	Stores final-completion audit decisions and supporting evidence when the auditor is enabled.
Memory retrieval trace	`planner_memory_debug/`	Stores retrieved structured memories used by the planner.
Failure diagnosis trace	`stuck_diagnosis/`	Stores failure attribution and suggested interventions when diagnosis is enabled.

verifier, and DONE auditor. By default, the verifier follows the main model unless a separate verifier model is configured, and the DONE auditor follows the verifier model. This keeps model routing explicit in the saved configuration instead of being hidden in prompt code.

Task splits and runtime configuration. OSWorld results use the OSWorld-Verified split described in Section 4.1. Mind2Web results use the protocol described in Appendix B. The Minecraft study follows the task setting described in Appendix F. Structured memory is constructed from the sources and indexing pipeline described in Appendix D. Runtime configuration records the task split, model aliases, step budget, screen size, memory-bank version, memory retrieval depth, and enabled verifier/auditor modules.

Logged artifacts. Each run writes a result directory containing both final outcomes and intermediate evidence. The top-level `args.json` records runtime configuration. Each task directory stores the final task result, screenshots, action traces, and module-specific debug files when the corresponding module is enabled. Table 13 summarizes the main artifacts used for auditing.

Scoring and aggregation. OSWorld scores are aggregated by the verified benchmark groups. Mind2Web results use the WebJudge-style evaluation protocol described in Appendix B. Minecraft metrics follow Appendix F. Aggregation is performed from saved result folders rather than from live agent decisions, so the reported scores can be traced back to the stored task outcomes and verification evidence.

Determinism and auditability. We fix task splits, display resolution, step budget, and runtime flags for each controlled comparison. GUI execution and remote or distributed VLM serving are not bit-exact across all reruns, so the release emphasizes matched configuration and auditability: each reported run should include the saved runtime configuration, task-level outcomes, screenshots, action traces, and verification logs needed to inspect the result.

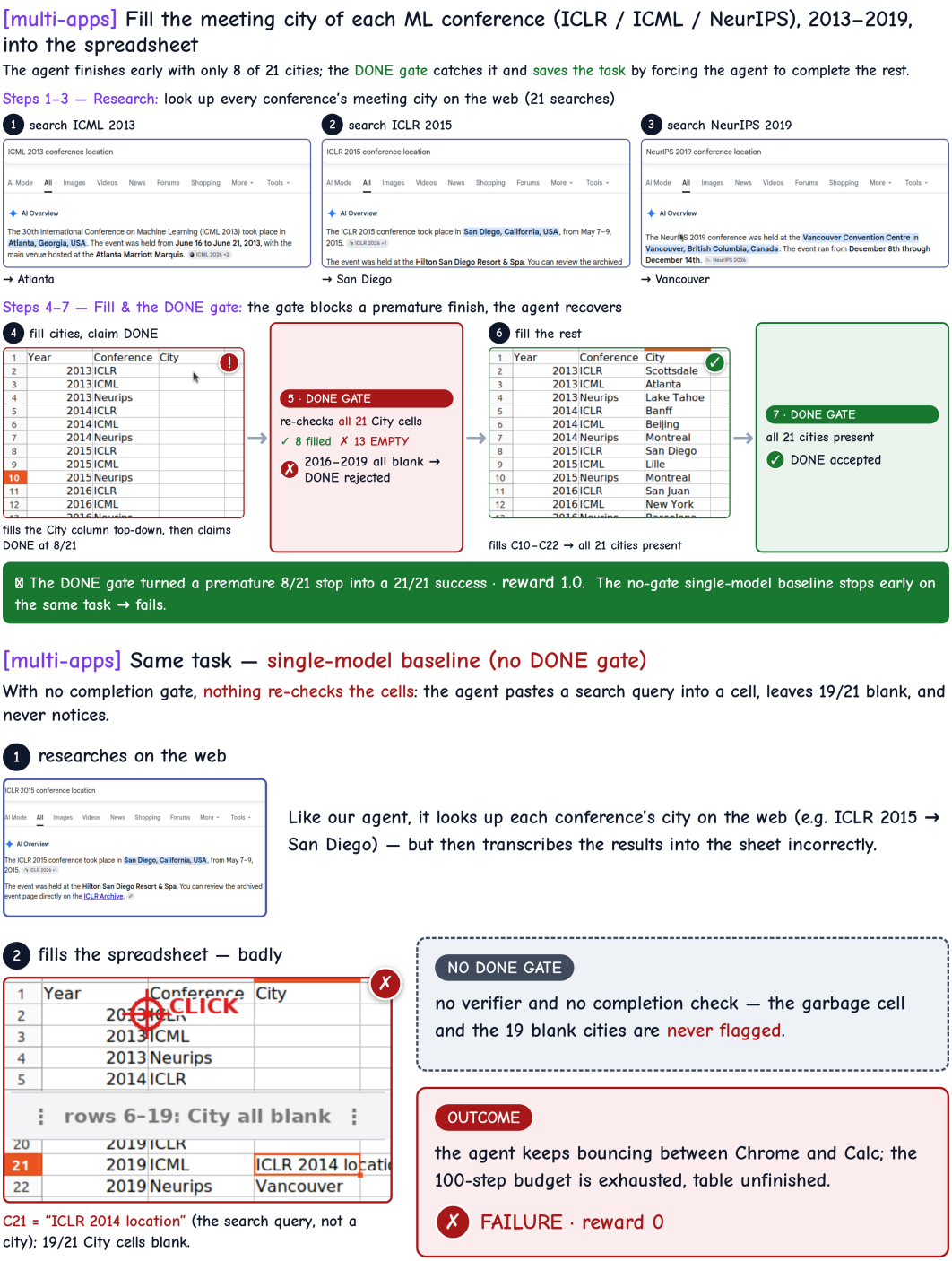


Figure 8: DONE auditing in a multi-app spreadsheet task. Top: the auditor rejects an early 8/21 completion claim and accepts the task only after all 21 city cells are filled. Bottom: without a DONE gate, the baseline never flags blank cells or a copied search query and exhausts the step budget.

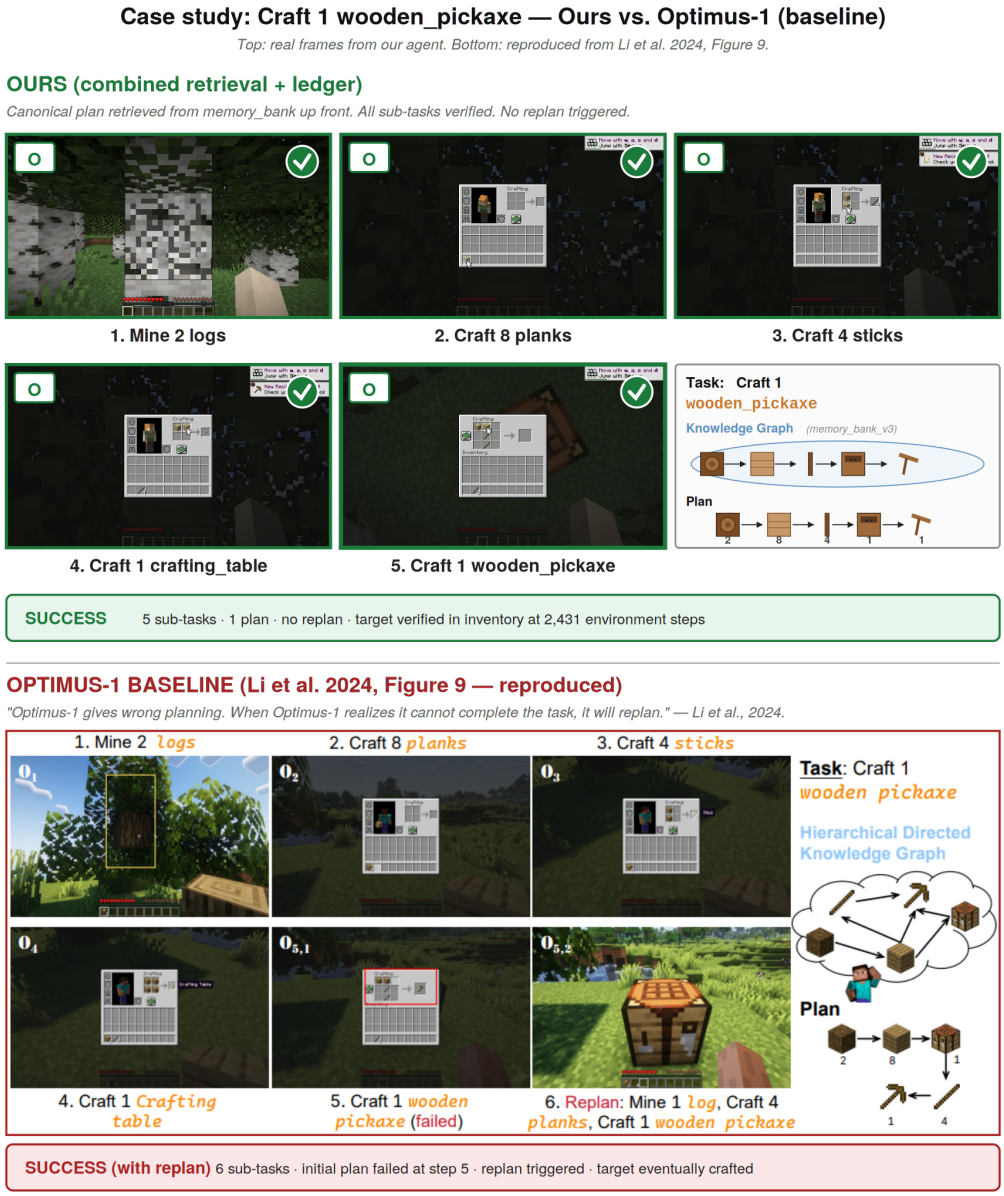


Figure 9: Case study on `wooden_pickaxe`. **Left:** our agent under combined retrieval + verification record. Real frames from the recorded trajectory; the canonical plan from `memory_bank_v3` occupies slot 6 (Task / Knowledge Graph / Plan). All five sub-tasks verify; no replan triggered; target produced at 2,431 environment steps. **Right:** Optimus-1 baseline, reproduced from Li et al. [15] Fig. 9; the planner proposes an under-resourced plan that fails at step 5 and recovers only via an in-flight replan.

H Prompts and Structured Output Contracts

This appendix reports the prompt templates and parsed contracts used by the LLM-facing modules. The full runtime prompts are composed from a static role prompt plus dynamic blocks, including the task, screenshots, accessibility tree, state, memory retrieval, verifier traces, and domain action catalogs. The excerpts below are copied from the implementation, with long catalogs and examples removed only where explicitly marked.

Table 14: LLM-facing prompt templates and contracts.

Module	Trigger	Output contract	Consumed by
Verification Requirements Initializer	Task start	JSON verification record seed with `initial_context`, `slots`, and `required_outcomes`.	Seeds the `VerificationRecord` and the first milestone verification specs.
Planner	Initial planning, progress check, and replan turns	XML-like tags for observation, subgoal assessment, decision, plan, and actor hint.	Updates the subgoal queue, strategy record, and the next actor instruction.
Actor	Low-level execution turn	JSON action array, normalized into UI-TARS-style [26] `Thought/Action` lines.	Compiled into GUI, host, or structured document actions.
Low-level verifier	After an actor turn within an execution loop	One JSON object with `status`, evidence, and optional actor feedback.	Decides whether actor control should continue, retry, or return to the planner.
Milestone verifier	Actor-loop exit and planner DONE boundary	JSON array of per-outcome judgments, optionally with one read-only probe.	Updates outcome states and proof traces in the verification record.
DONE auditor	After the structured DONE gate accepts a planner DONE claim	Structured text ending with `VERDICT: PASS|FAIL|PARTIAL`.	Final adversarial check before accepting task completion.
Failure attribution	Repeated stuck or rejected progress	JSON diagnosis with role attribution and intervention fields.	Routed to planner replan, actor redo, verifier override, environment intervention, or fallback.
BBON judge	Section 5 analysis only	Per-candidate verdict lines plus `<choice>` and `<cite>`.	Selects one rollout among multiple candidates for test-time analysis.

Verification Requirement Initializer. At task start, the initializer receives the user instruction, the initial screenshot, the accessibility tree, and environment-derived context. It returns a JSON seed for the verification record. The prompt first asks the model to separate durable milestones from transient waypoints:

Verification Requirement initializer prompt excerpt

```
You initialize a structured progress verification record for a GUI
automation agent. The agent runs inside an OSWorld benchmark VM.

You see the user's task instruction and the agent's initial screen.
Your job is to read them and output:

  1. initial_context -- what the agent is starting with (app, active URL,
     window title, visible open tabs). Read these off the screen and the
     provided a11y tree. If you cannot read a field, leave it null.

  2. required_outcomes -- a small ORDERED chain of GATING MILESTONES that
     the agent must pass through to complete the task. NOT a checklist
     of every screen interaction; the planner breaks down the
     micro-steps.

Test: "If this state regressed, would the AGENT need to redo
OTHER work too -- i.e., does it CASCADE downstream?"

  YES -> MILESTONE   (include as outcome)
  NO  -> WAYPOINT    (DROP -- let the planner handle as subgoal)
```

The abridged schema below matches the implementation contract; omitted fields are optional fields of the same `verify` object.

Verification Requirements output contract

```
{
  "initial_context": {
    "open_tabs": ["<url>", ...],
    "active_url": "<url or null>",
    "active_app": "<app name or null>",
    "window_title": "<title or null>"
  },
  "value_provenance": [
    {"value": "...", "status": "KNOWN|UNKNOWN", "why": "..."}
  ],
  "slots": [
    {"name": "...", "type": "url|text|path|numeric|cell_ref",
     "description": "...", "source_hint": "..."}
  ],
  "required_outcomes": [
    {
      "id": "<snake_case>",
      "description": "<one sentence>",
      "evidence_hint": "<concrete observable>",
      "depends_on": ["<other_outcome_id>", ...],
      "verify": {
        "kind": "<one verify kind>",
        "file_path": "...",
        "patterns": ["<regex>", ...],
        "url_pattern": "...",
        "tag": "...",
        "text_contains": ["...", ...],
        "state_contains": ["...", ...],
        "visual_must_hold": ["...", ...],
        "command": ["<argv>", ...],
        "expected_substring": ["...", ...],
        "expected_exit_code": 0,
        "calc_checks": [{"op": "<op>", "...": "..."}],
        "impress_checks": [{"op": "<op>", "...": "..."}],
        "writer_checks": [{"op": "<op>", "...": "..."}]
      }
    }
  ]
}
```

Planner. The planner uses separate prompt builders for initial planning, progress checking, and forced replanning, but the recurrent parser reads the same core tags. The planner first describes observations and assesses the latest subgoal, then emits a decision. A `REPLAN` decision may replace the queue with a new `plan`; `CONTINUE` keeps the current queue; `DONE` enters the structured DONE gate.

Planner prompt excerpt

```
Output format -- emit these tags in EXACTLY this order:

<observations>
  <visible>CURRENT SCREEN -- the app / window / page in focus
  ... plus the INTERACTIVE elements you can act on right now.</visible>
  <unexplored>Content the task may need that is NOT in the
  current view.</unexplored>
</observations>

<last_subgoal_assessment>
  <evidence>One short sentence citing CONCRETE evidence that
  the previous subgoal's expected_post_state is -- or is not --
  satisfied.</evidence>
  <subgoal_anchor_check>...</subgoal_anchor_check>
  <contradiction_check>...</contradiction_check>
  <done>YES or NO</done>
</last_subgoal_assessment>

<Bottleneck>...</Bottleneck>
<decision>DONE|CONTINUE|REPLAN|INFEASIBLE</decision>
<plan>
  <strategy>...</strategy>
  <step>
    <text>...</text>
```

```
    <expected_post_state>...</expected_post_state>
  </step>
</plan>
<actor_hint>...</actor_hint>
```

The planner prompt also binds decisions to runtime effects. In the live prompt, `DONE` requires every required outcome to be verified, the remaining subgoal list to be empty, and no contradiction between verification record state and visible state. This is what turns planner text into controlled state advancement rather than a free-form completion claim.

Actor. The actor decomposes the current subgoal into a JSON array. Its base prompt and the inline grounding directive are appended together at runtime:

Actor prompt excerpt

```
You are a UI action decomposer. Given the current screenshot and a
subgoal, emit a JSON array of low-level actions to execute in order.
You output the on-screen pixel location of each element YOURSELF ...

STRICT FORMAT RULES -- violate any and downstream parsing fails:
- Respond with EXACTLY ONE JSON array. No prose before or after.
  No markdown fences (no ‘‘‘json).
- Use double quotes, not single quotes, for every JSON string.
- Every "target" / "start_target" / "end_target" / "anchor_target"
  MUST be a description that pinpoints exactly one element visible in
  the screenshot ... you then give its coordinates yourself.
- GROUNDING-VISIBILITY RULE: every target ... is grounded against the
  CURRENT screenshot -- the one attached to this prompt -- not against
  some imagined future screenshot.
```

Inline grounding directive

```
[INLINE GROUNDING MODE]
For EVERY action that targets a UI element, in ADDITION to the
existing "target" text field you MUST include the element’s location
read directly from the screenshot, as coordinates normalized to a
0-1000 grid:
  - click / left_double / right_single: add "point": [x, y]
  - drag: add "start_point": [x, y] and "end_point": [x, y]
  - scroll: add "anchor_point": [x, y]
[omitted: non-spatial action list]
Example: {"action": "click", "target": "the Submit button",
"point": [840, 560]}
```

The parsed action contract used in the paper experiments is:

Actor parsed action contract

```
[
  {"action": "click|left_double|right_single",
   "target": "<visible element>", "point": [x, y]},
  {"action": "drag",
   "start_target": "...", "end_target": "...",
   "start_point": [x, y], "end_point": [x, y]},
  {"action": "type",
   "text": "...", "wipe_then_type": false, "submit": false},
  {"action": "hotkey", "key": "..."},
  {"action": "scroll",
   "direction": "up|down|left|right",
   "anchor_target": "...", "anchor_point": [x, y]},
  {"action": "wait"},
  {"action": "impossible", "reason": "..."}
]
```

Host-side and structured-document actions such as `open_app`, `navigate`, `extract_info`, `cli_run`, `edit_json`, `calc_*`, `impress_*`, and `writer_*` are injected only when available for the current domain.

Low-level verifier. The low-level verifier judges only the current subgoal after one actor turn. Its input includes the task, subgoal, expected post-state, actor response, executed actions, before/after screenshots, up to three prior screenshots, and the recent actor-action history. It does not update the verification record.

Low-level verifier prompt excerpt

```
You are a strict low-level verifier for a computer-use agent.

Judge only the CURRENT SUBGOAL, not the whole task. Use the recent
actor-turn history for context, then judge the CURRENT actor turn by
comparing the CURRENT BEFORE and CURRENT AFTER screenshots with the
actor's last action.

Return:
- complete: the current subgoal is visibly satisfied now.
- continue: the action made useful progress, but the subgoal is not complete.
- no_effect: the action appears to have no useful visible effect.
- off_track: the agent moved to the wrong app/page/dialog or changed
  the wrong thing.
- stuck: the screen is blocked, loading, errored, or needs higher-level
  replanning.

Be conservative. Do not mark complete unless the AFTER screenshot clearly
supports the expected post-state. Output JSON only.
```

Low-level verifier output contract

```
{
  "status": "complete|continue|no_effect|off_track|stuck",
  "reason": "one short sentence",
  "evidence": "brief visible evidence from the AFTER screenshot",
  "feedback_to_actor": "specific next low-level hint, or null"
}
```

Milestone verifier. The milestone verifier runs over target outcomes at a boundary. It first judges from observation and may request one read-only deterministic probe when the success condition is latent. A valid probe overrides the visible judgment; an invalid or inconclusive probe falls back to the observation judgment. Anything other than an explicit `verified` keeps the outcome unverified.

Milestone verifier prompt excerpt

```
You are the MILESTONE VERIFIER for a GUI computer-use agent. The planner
just declared a subgoal complete. For each listed milestone, decide
whether it is OBJECTIVELY achieved on the CURRENT screen -- reason from
evidence; never rubber-stamp the planner's claim.

THE TASK INSTRUCTION IS THE GROUND TRUTH -- NOT THE MILESTONE LABEL.

LATENT STATE -> PROBE. A screenshot cannot prove a file's saved content,
a document's model (cell values / fonts / shapes), or the active URL.
When a milestone hinges on latent state, request ONE read-only check
rather than guessing from pixels.

NO EVIDENCE -> NOT VERIFIED. If you cannot quote concrete current evidence
that EVERY constraint holds, the milestone is not verified.
```

Milestone verifier output contract

```
[
  {
    "outcome_id": "<id>",
    "constraints": "<task-specific requirements>",
    "observation": "<current a11y/screen/doc evidence>",
    "obs_verdict": "verified|not_verified|uncertain",
    "reason": "...",
    "evidence": "...",
```

```
    "probe": null
  }
]
```

The allowed probe kinds are `file_grep`, `url_match`, `a11y_match`, `calc_verify`, `impress_verify`, `writer_verify`, and guarded read-only `shell_command`. The available kinds are filtered by domain before they are shown to the verifier.

DONE auditor. The DONE auditor is a final audit, not another JSON verifier. It receives a snapshot with the task text, current accessibility tree, current URL, verifier specs, accepted proof claims, perceiver focus block, and recent screenshots. It must parse the task first, cross-check evidence, list gaps when present, and end with a single verdict line.

DONE auditor prompt excerpt

```
You are the Done-Auditor for an OSWorld GUI agent. The agent has just
claimed the task is DONE. A separate structured verifier has already
approved this claim against its own checklist. Your job is to
adversarially refute that approval by examining only the raw evidence
you receive -- the user's literal task text, the screenshots you see,
the accessibility tree, and the proof traces the structured verifier
already accepted.

You do NOT see the planner's chain of thought, the plan steps, the
current subgoal text, or any strategy notes. This is by design -- you
are a fresh pair of eyes.

TASK TEXT IS TRUTH; verifier_specs are ONE LLM's guess at success
written at task start. When the spec marks an outcome PASS but the
task text names a target ... trust the task text and FAIL.
```

DONE auditor output contract

```
TASK PARSE:
  <one bullet per required deliverable>

EVIDENCE CROSS-CHECK:
  <one bullet per task requirement with evidence>

GAP ANALYSIS (if any):
  <one bullet per gap>

VERDICT: PASS|FAIL|PARTIAL
```

Failure attribution and intervention. Failure attribution is invoked when the agent repeatedly fails to make verified progress. The schema forces the model to recall relevant memory, observe the current state, critique divergence from memory, attribute the failure to a role, and propose an intervention. The downstream router consumes the role and role-specific payloads rather than treating the diagnosis as free text.

Failure attribution prompt excerpt

```
You are a root-cause failure-attribution analyst for a GUI agent stuck
on a single goal. You produce ONE JSON object -- nothing else. No prose
before or after. No markdown headings. No "Phase 1" labels. Just the JSON.

JSON field requirements (fill in this exact order so the conclusion is
grounded in earlier evidence):

1. memory_recall.canonical_approach: one sentence summarising what
RETRIEVED MEMORY recommends for this goal.
2. memory_recall.memory_referenced: array of 1-4 memory entries you
actually relied on.
3. observation_summary: 2 sentences MAX -- what the agent did, what the
current state actually shows.
```

```
4. evidence_quotes: array of 2-4 verbatim strings from the snapshot ...
Every claim downstream MUST be anchored to one of these.
```

Failure attribution output contract

```
{
  "memory_recall": {
    "canonical_approach": "...",
    "memory_referenced": [
      {"layer": "L1|L2|L3a|L3c", "id": "...",
       "score": 0.0, "why_relevant": "..."}
    ]
  },
  "observation_summary": "...",
  "evidence_quotes": ["...", "..."],
  "memory_critique": {
    "memory_followed": "true|false|partial|n/a",
    "divergence_description": "..."
  },
  "root_cause_summary": "...",
  "attributed_to_role": "planner|actor|verifier|env|unclear",
  "category": "tactical|strategic|info_gap|unclear",
  "tactical_subkind": "<one tactical subkind or null>",
  "missing_or_wrong": ["..."],
  "confidence": "high|medium|low",
  "next_action_hint": "...",
  "verifier_override_evidence": [],
  "env_recovery_kind": null,
  "env_recovery_params": {}
}
```

Valid `tactical_subkind` values are `wrong_target`, `target_missing`, `target_blocked`, `blocking_state`, `focus_lost`, and `execution_error`; otherwise the value is `null`. The router maps this diagnosis to one of five intervention decisions: `planner_replan`, `actor_redo`, `verifier_override`, `env_intervention`, or `fallback_planner`. Safety gates keep role-specific interventions from firing when the evidence is weak.

BBON judge. BBON is used only in the analysis section. The judge receives a shared success rubric and several candidate behavior narratives with decisive screenshots. It first emits one verdict line per candidate, then chooses a single candidate.

BBON judge prompt excerpt

```
You are an exacting judge selecting which candidate ACTUALLY completed a
computer-use task. You get a SUCCESS RUBRIC (the criteria) and, per
candidate, a behavior storyboard: objective actions + observed evidence
+ screenshots at the decisive steps, and the FINAL STATE.

METHOD:
  - For EACH rubric outcome, decide which candidates satisfy it FROM THE
EVIDENCE -- weight the FINAL STATE most. Do NOT trust any 'AGENT
SELF-CLAIM' line; agents frequently believe they succeeded when they did not.
  - Watch for an outcome reached then BROKEN later ...
  - Then pick the SINGLE candidate that best satisfies ALL outcomes.
```

BBON judge output contract

```
<verdict>A: DONE|NOT_DONE -- evidence for candidate A</verdict>
<verdict>B: DONE|NOT_DONE -- evidence for candidate B</verdict>
...
<choice>A</choice><cite>one sentence citing decisive evidence</cite>
```

Parsing and fallbacks. All contracts are parsed defensively. Planner tags tolerate missing optional blocks. Actor JSON is retried once if unparseable, then falls back to `<Impossible>`. Low-level verifier parse failures default to `continue`. Milestone verifier failures keep outcomes unverified, and

malformed probes are ignored. DONE-auditor transport or parse failures map to `PARTIAL`. BBON choices are retried on invalid `<choice>` output before the analysis code falls back and marks the parse failure.